中山大學 SUN YAT-SEN UNIVERSITY

国家超级计算广州中心 NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

DCS290

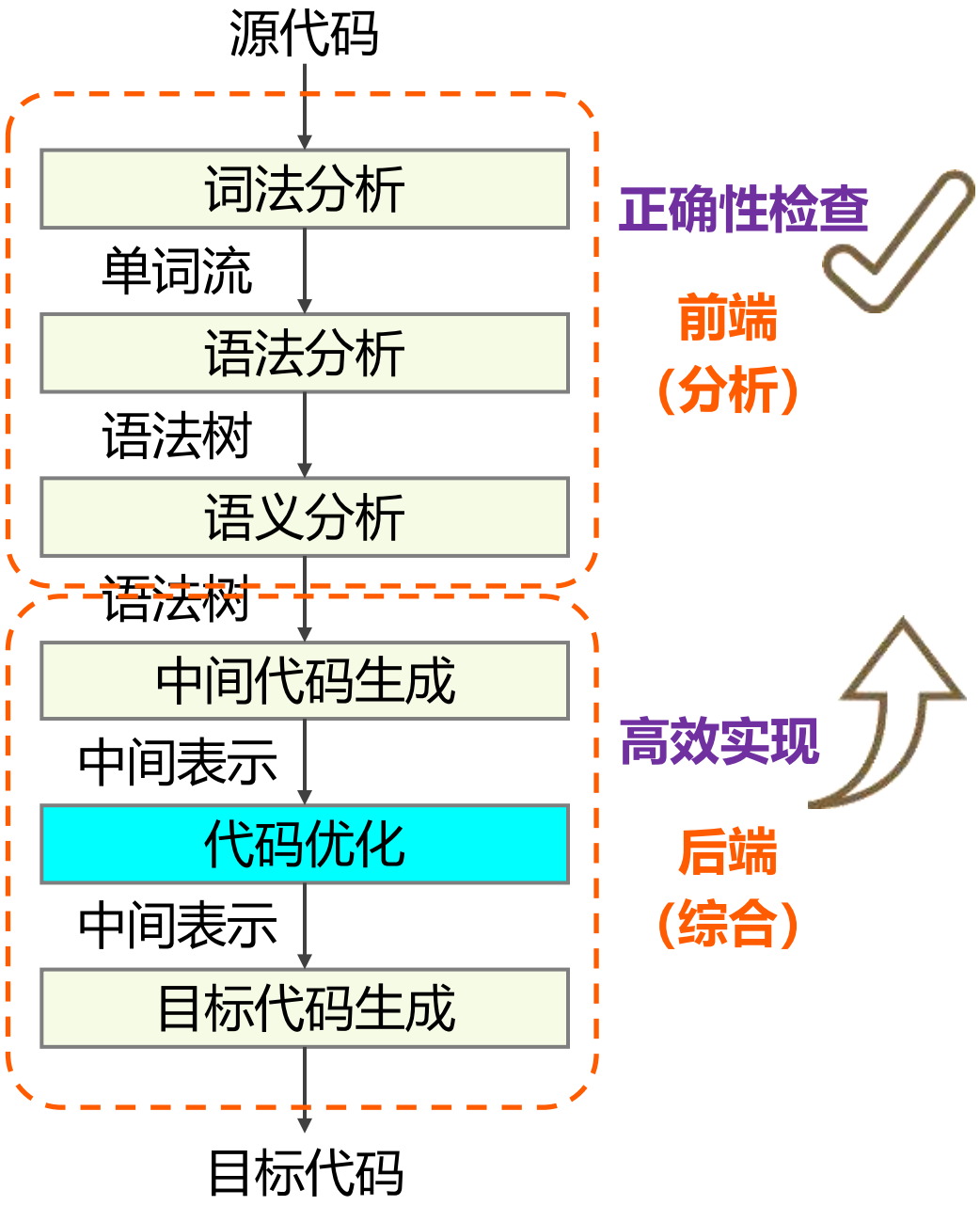
Compilation Principle

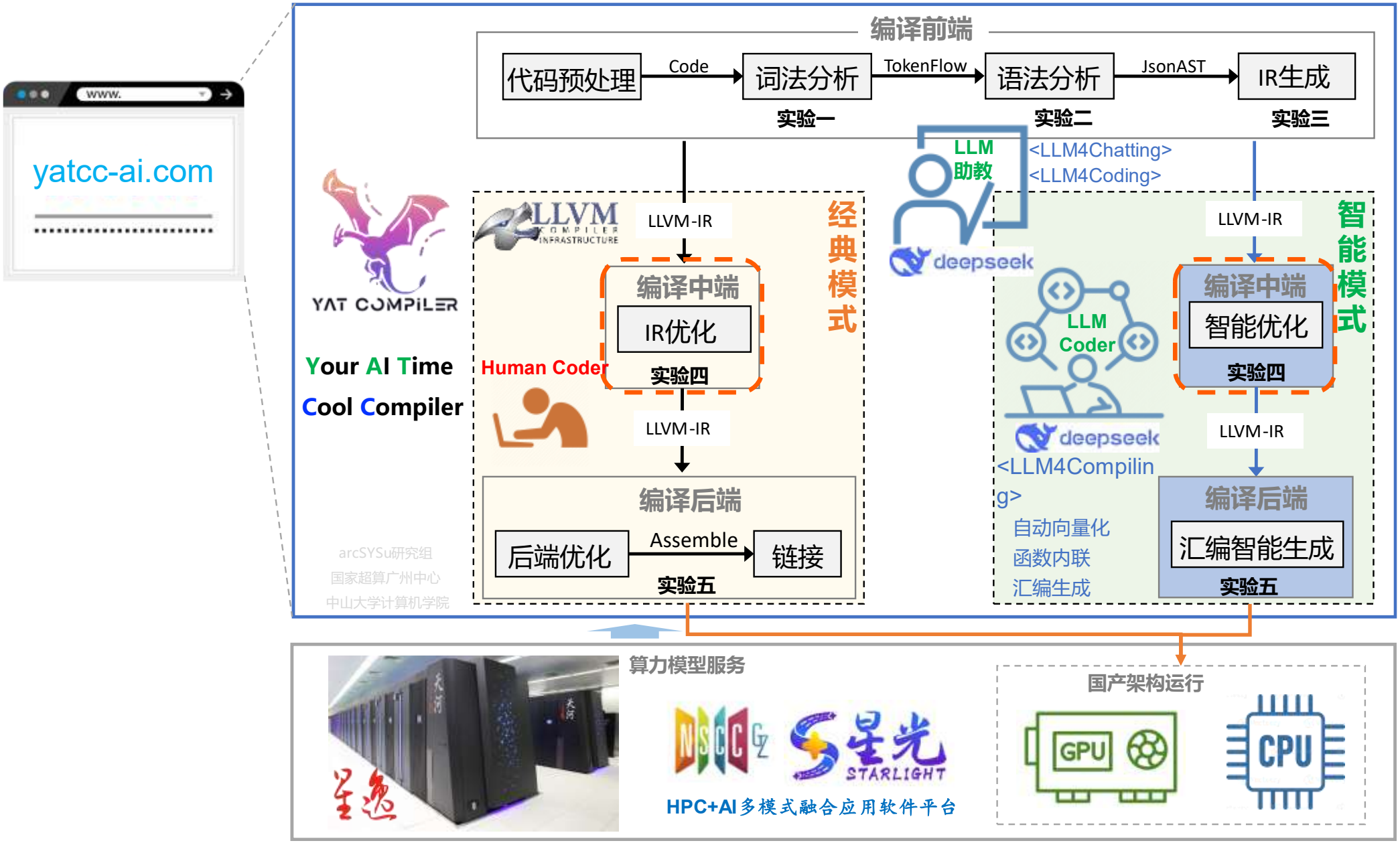
编 译 原 理

第九章 代码优化

郑馥丹

zhengfd5@mail.sysu.edu.cn


源代码
词法分析
单词流
语法分析
语法树
语义分析
语法树
中间代码生成
中间表示
代码优化
中间表示
目标代码生成
目标代码
正确性检查
前端
（分析）
高效实现
后端
（综合）


www.
yatcc-ai.com
编译前端
代码预处理
Code
词法分析
TokenFlow
语法分析
JsonAST
IR生成
实验一
实验二
实验三
LLM
助教
<LLM4Chatting>
<LLM4Coding>
deepseek
YAT COMPILER
Your AI Time
Cool Compiler
LLVM
COMPILER
INFRASTRUCTURE
LLVM-IR
经典模式
编译中端
IR优化
实验四
Human Coder
LLVM-IR
编译后端
后端优化
Assemble
链接
实验五
arcSYSu研究组
国家超算广州中心
中山大学计算机学院
LLVM-IR
智能模式
编译中端
智能优化
实验四
LLM
Coder
deepseek
LLVM-IR
<LLM4Compiling>
自动向量化
函数内联
汇编生成
编译后端
汇编智能生成
实验五
算力模型服务
NSCC GZ
星光
STARLIGHT
HPC+AI多模式融合应用软件平台
国产架构运行
GPU
CPU

CONTENTS

目 录

01 代码优化简介 Introduction

02 局部优化 Local Optimization

03 循环优化 Loop Optimization

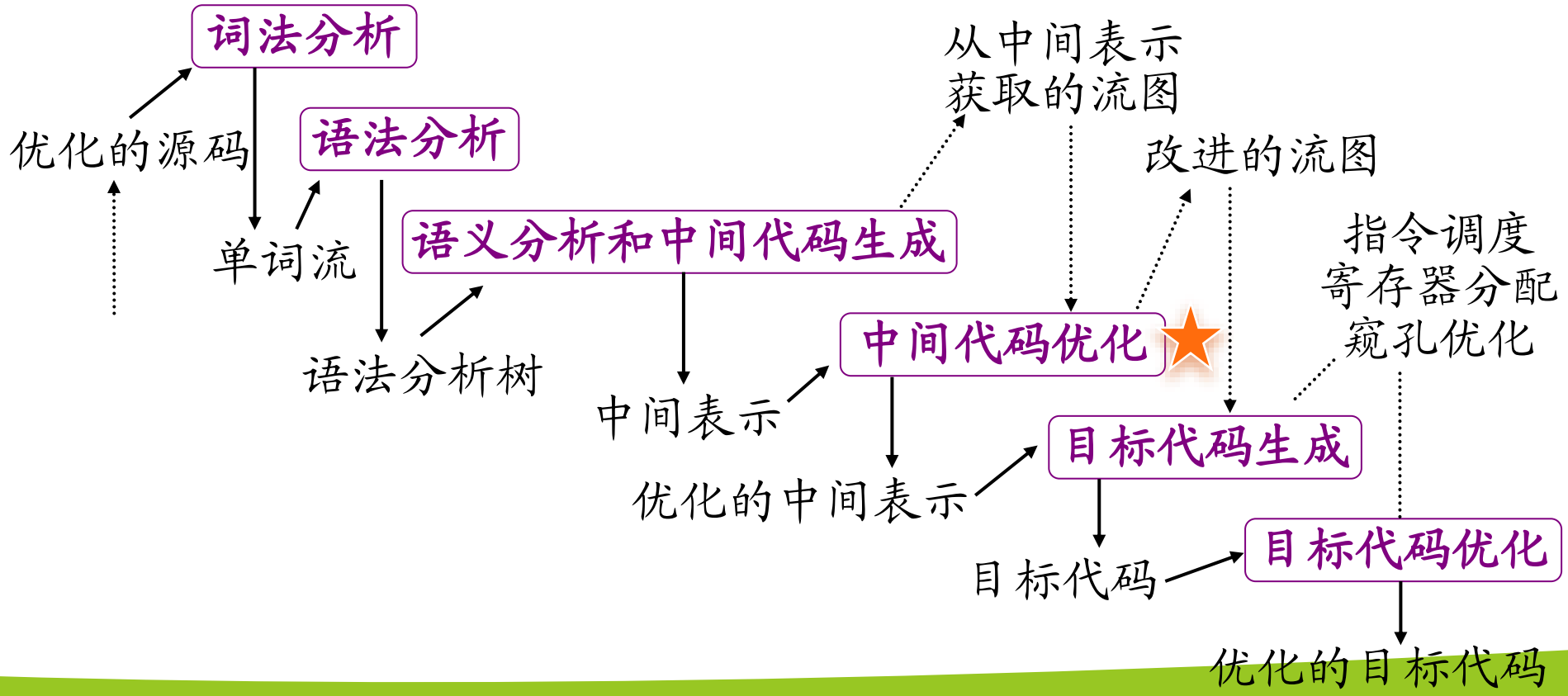
04 全局优化 Global Optimization

1. 定义

- 什么是代码优化[Code Optimization]
  - 所谓优化是对代码进行**等价变换**，使得变换后的代码的**效率更高**（节省运行时间、存储空间或两者兼而有之）
  - **语义保持转换**[Semantics-preserving Transformations]
  - 代码优化 vs. 代码改进[Code Improvement]
- 权衡[Trade-off]
  - **时间效率 vs.空间效率**
  - **编译器效率 vs.目标代码效率**
    - ✓ 高级别的优化→高效的目标代码 ⇿ 更多的编译时间
    - ✓ 复杂的优化 ⇿ 更多编译内存占用

2. 优化级别

- 三种优化级别
  - 源代码优化
  - **中间代码优化**（不依赖具体计算机）★
  - 目标代码优化（依赖于具体计算机）



3. 优化的分类

- 根据优化涉及的程序范围，分为：
  - **局部优化[Local Optimization]**
    - ✓ 在只有一个入口、一个出口的**基本块[basic block]**上进行优化
  - **循环优化[Loop Optimization]**
    - ✓ 对循环中的代码进行优化
    - ✓ 基于控制流分析
  - **全局优化[Global Optimization]**
    - ✓ 在整个程序范围内进行的优化
    - ✓ 基于数据流分析
  - 窥孔优化[Peephole Optimization]
    - ✓ 对一个滑动窗口内的代码进行优化

4. 通用技术

(1) **删除公共子表达式**[Common Subexpressions]

– 若子表达式E在前面计算过，且之后E中的**变量值都未改变**，则E的重复出现称为公共子表达式，可将其删除，避免重复计算

(1) T_1 :=4*I

(2) T_2 :=addr(A) - 4

(3) T_3 :=T_2[T_1]

(4) T_4 :=4*I ⇒ (4) T_4 :=T_1

(5) ……

4. 通用技术

(2) **常量折叠及复写传播**[Copy Propagation]

– 如果运算量都是已知量，则在编译时就算出它的值，称为**常量折叠（合并已知量）**。

– 若有A:=B，称为把B值复写到A。如果其后有引用A的地方，且其间A、B的值都未改变，则可把对A的引用改为对B引用，称为**复写传播**。

(1) I:=1

(2) T_1:=4*I ⇒ (2) T_1:=4 常量折叠

(3) T_4:=T_1 ⇒ (3) T_4:=4 常量传播

(4) T_6:=T_5[T_4] ⇒ (4) T_6:=T_5[T_1] 复写传播

4. 通用技术

(3) 消除死代码（删除无用赋值）

– 死代码/无用赋值：

- ✓ 变量被赋值，但在程序中从未被引用
- ✓ 变量赋值后未被引用又重新赋值，则前面赋值是无用的
- ✓ 变量的赋值仅被计算变量自己引用

(1) I:=1
(2) T_1:=4
(3) T_3:=T_2[T_1]
(4) T_4:=T_1
(5) I:=I+1
(6) T_1:=T_1+4
(7) if T_1≤80 goto (3)



(2) T_1:=4
(3) T_3:=T_2 [T_1]
(6) T_1:=T_1+4
(7) if T_1≤80 goto (3)

CONTENTS

目 录

01 代码优化简介 Introduction

02 局部优化 Local Optimization

03 循环优化 Loop Optimization

04 全局优化 Global Optimization

1. 定义

- 局部优化是指**基本块内**的优化
  - **基本块**是指程序中一个顺序执行的语句序列，其中**只有一个入口语句和一个出口语句**。执行时只能从入口语句进入，从其出口语句退出。

- 局部优化的逻辑：



2. 基本块的划分

(1) 求基本块的入口语句，它们是：

– 程序的**第一个语句**；

– 条件转移或无条件转移语句的**转移目标语句**；

– 紧跟在**条件转移语句后面的语句**。

```
(1) read  X
(2) read  Y
(3) R:=X  mod  Y
(4) if  R=0   goto  (8)
(5) X:=Y
(6) Y:=R
(7) goto  (3)
(8) write  Y
(9) halt
```

入口语句：
(1)、(3)、(5)、(8)

2. 基本块的划分

(2) 对每一入口语句，构造其所属的基本块:

– 它是由该入口语句到**下一入口语句**(不包括下一入口语句);

– 或到**一转移语句**(包括该转移语句);

– 或到**一停止语句**(包括该停止语句)之间的语句序列组成的。

```
(1) read  X
(2) read  Y
(3) R:=X  mod  Y
(4) if  R=0   goto  (8)
(5) X:=Y
(6) Y:=R
(7) goto  (3)
(8) write  Y
(9) halt
```

入口语句:
(1)、(3)、(5)、(8)
分别构成基本块:
B1 {(1)、(2)}
B2 {(3)、(4)}
B3 {(5)、(6)、(7)}
B4 {(8)、(9)}

凡未被纳入某一基本块的语句，是不会被执行到的语句，可删除!

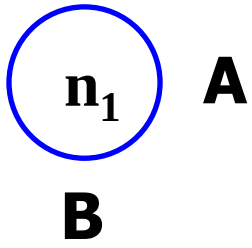
3. 基本块的DAG表示

- **DAG[Directed Acyclic Graph]**：有向无环图
- 基本块的DAG是一种结点带有标记或附加信息的DAG：
  - **叶子结点：标识符或常数**，代表变量或常数的值
  - **内部结点：运算符**，代表用该算符对其后继结点所代表的值进行运算的结果
  - 各结点都可以附加上一个或多个标识符，表示这些变量具有该结点所代表的值

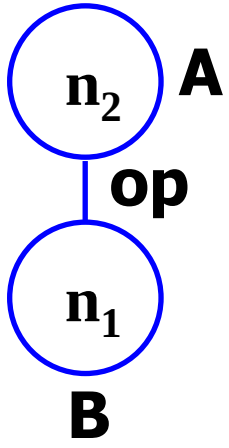
3. 基本块的DAG表示

- 具体结点形式

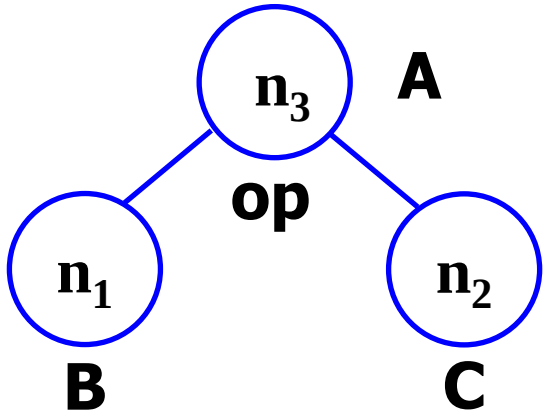
0型： A:=B　　(:=, B, －,A)



1型：
A:=op B　(op , B, －,A)

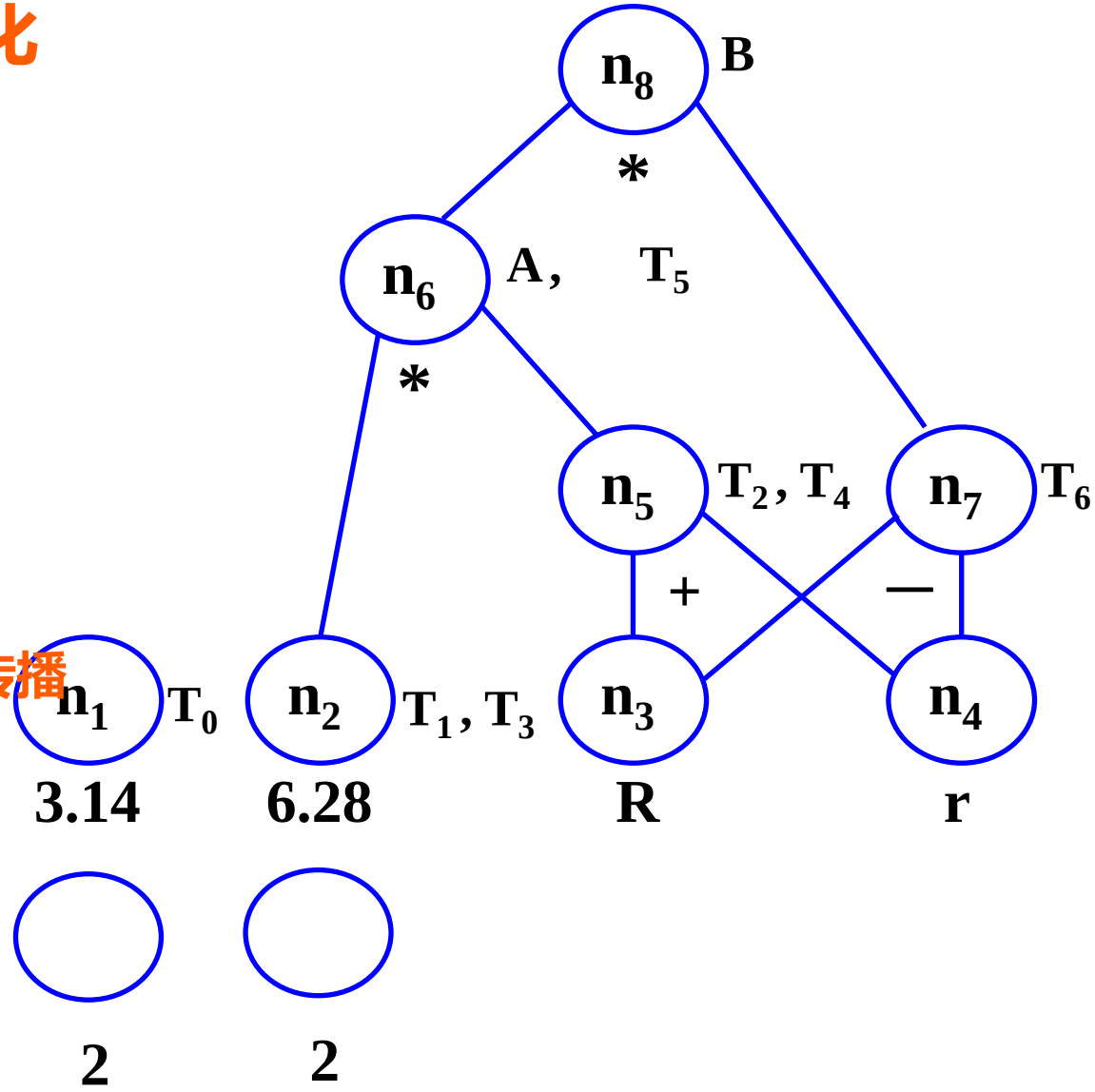


2型：
A:=B op C　(op , B , C , A)



4. DAG的应用

- **例：构造以下基本块的DAG并进行优化**
  - (1) T_0:=3.14
  - ~~(2) T_1:=2*T_0~~ T_1 :=6.28 常量折叠
  - (3) T_2:=R+r
  - (4) A:=T_1*T_2
  - ~~(5) B:=A~~ 消除死代码
  - ~~(6) T_3:=2*T_0~~ T_3 :=6.28 常量折叠+常量传播
  - ~~(7) T_4:=R+r~~ T_4 := T_2 删除公共子表达式
  - ~~(8) T_5:=T_3*T_4~~ T_5 := A 删除公共子表达式
  - (9) T_6:=R—r
  - ~~(10) B:=T_5*T_6~~ B := A* T_6 复写传播

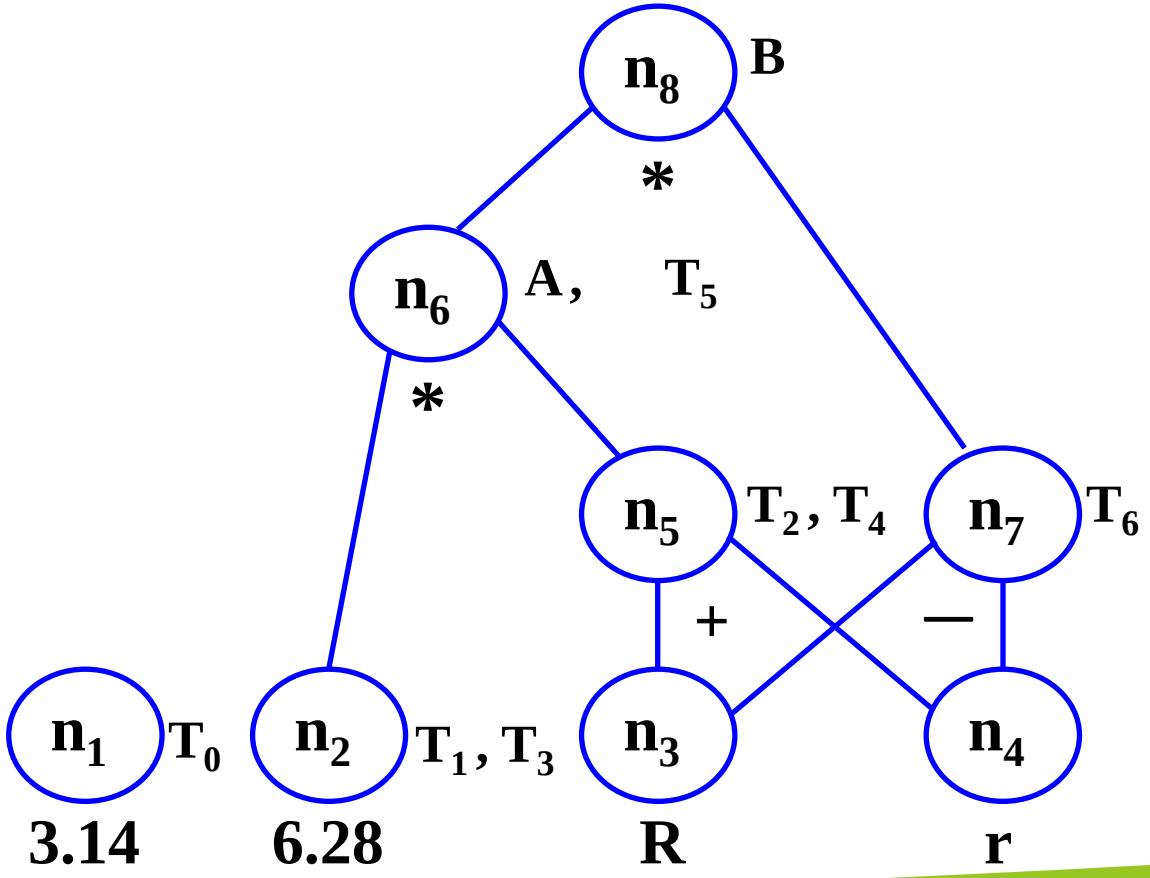


4. DAG的应用

- 利用DAG进行优化
  - 删除在基本块后不被引用变量的赋值

假如T_0，T_1，…，T_6在基本块后都不被引用，则这些符号可在DAG附加标识符中删去

(1) T_0 :=3.14
(2) T_1 :=6.28
(3) T_3 :=6.28
(4) T_2 :=R+r
(5) T_4 :=T_2
(6) A :=6.28*T_2
(7) T_5 :=A
(8) T_6 :=R−r
(9) B :=A*T_6

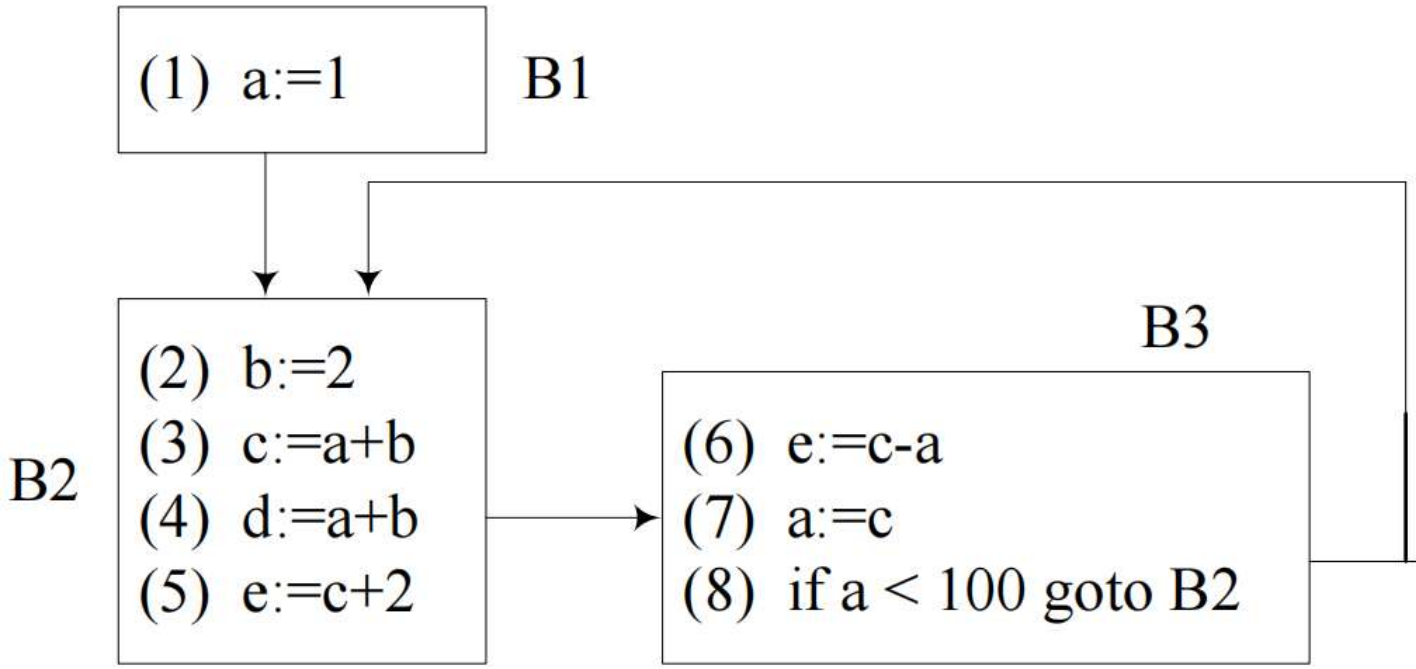


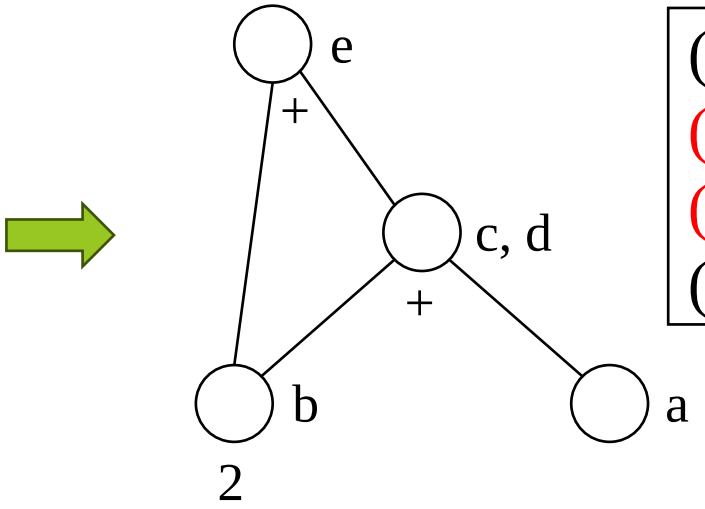
随堂练习（1）

对于以下流图：

(1) 为基本块B2构造DAG图；

(2) 根据DAG图优化基本块B2。





(2) b:=2
(3) c:=a+2
(4) d:=c
(5) e:=c+2

CONTENTS

目 录

01 代码优化简介 Introduction

02 局部优化 Local Optimization

03 循环优化 Loop Optimization

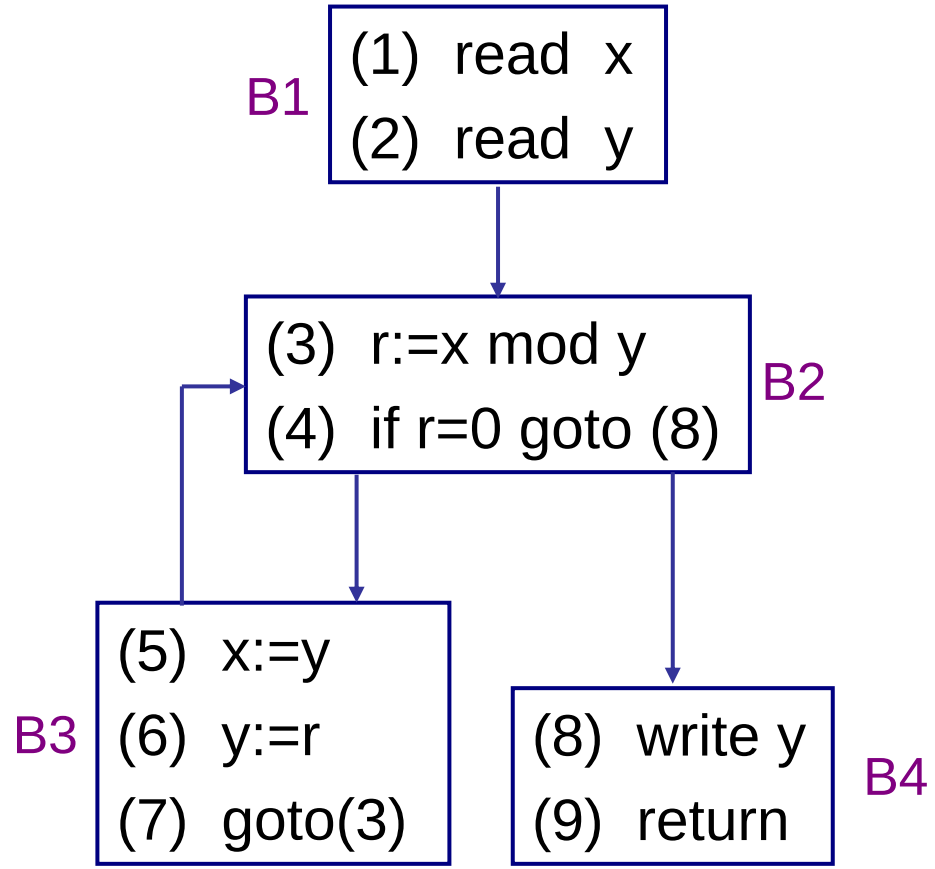
04 全局优化 Global Optimization

1. 简介

- **循环优化对提高目标代码的效率作用显著**
- 基于**流图[Flow Graphs]**进行
  - 把**控制流的信息**加到基本块集合上构成的**有向图**称为表示程序的流图，简称流图。
  - 流图的构造方法：
    - ✓ 点集：以基本块为结点，含程序第一条语句的结点为首结点。
    - ✓ 边集：从基本块B_i向基本块B_j引有向边，仅当
      - **B_j在程序中的位置紧跟在B_i之后**，且B_i的出口语句不是无条件转移语句或停止语句。或者
      - **B_i的出口**是转移语句（goto（s)或if...goto（s)），并且**转移目标(s)是B_j的入口**语句。

1. 简介

```
*(1)  read  x
  (2)  read y
*(3)  r:=x mod y
  (4)  if r=0 goto (8)
*(5)  x:=y
  (6)  y:=r
  (7)  goto(3)
*(8)  write y
  (9)  return
```



2. 优化技术

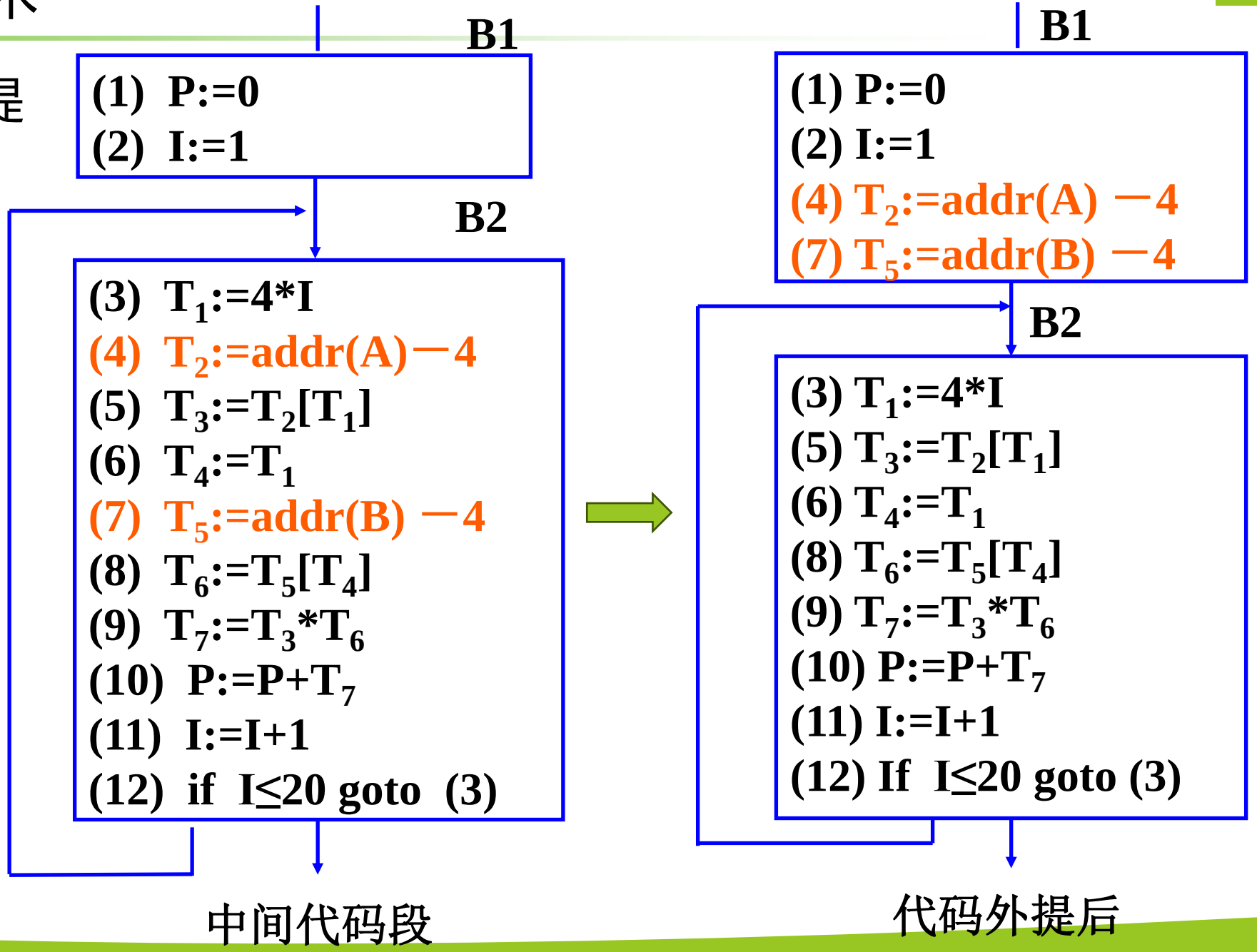
(1) 代码外提[Code Motion]

– 把**循环不变运算**提到循环的前面，使之只在循环外计算一次。

– 循环不变运算：其结果独立于循环执行次数的表达式；运算量为常量或在循环外定值，**每次循环时其值不变的运算**。

2. 优化技术

(1) 代码外提


B1
(1) P:=0
(2) I:=1
B2
(3) T_1:=4*I
(4) T_2:=addr(A)－4
(5) T_3:=T_2[T_1]
(6) T_4:=T_1
(7) T_5:=addr(B) －4
(8) T_6:=T_5[T_4]
(9) T_7:=T_3*T_6
(10) P:=P+T_7
(11) I:=I+1
(12) if I≤20 goto (3)
中间代码段
B1
(1) P:=0
(2) I:=1
(4) T_2:=addr(A) －4
(7) T_5:=addr(B) －4
B2
(3) T_1:=4*I
(5) T_3:=T_2[T_1]
(6) T_4:=T_1
(8) T_6:=T_5[T_4]
(9) T_7:=T_3*T_6
(10) P:=P+T_7
(11) I:=I+1
(12) If I≤20 goto (3)
代码外提后

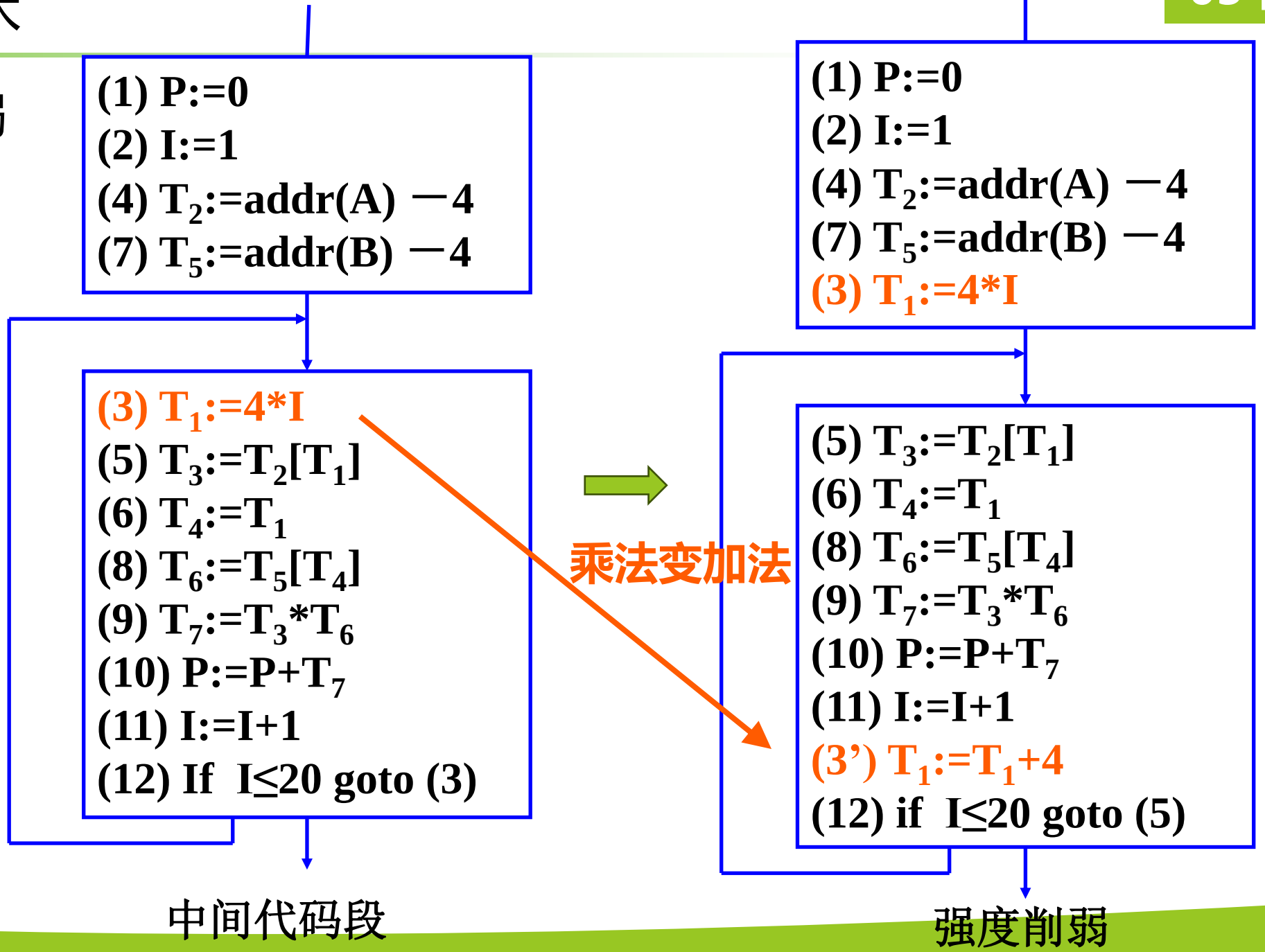
2. 优化技术

(2) 强度削弱[Reduction in Strength]

– 把程序中**强度大的运算替换成强度小**的运算。例如把乘法运算换成加法运算等。

2. 优化技术

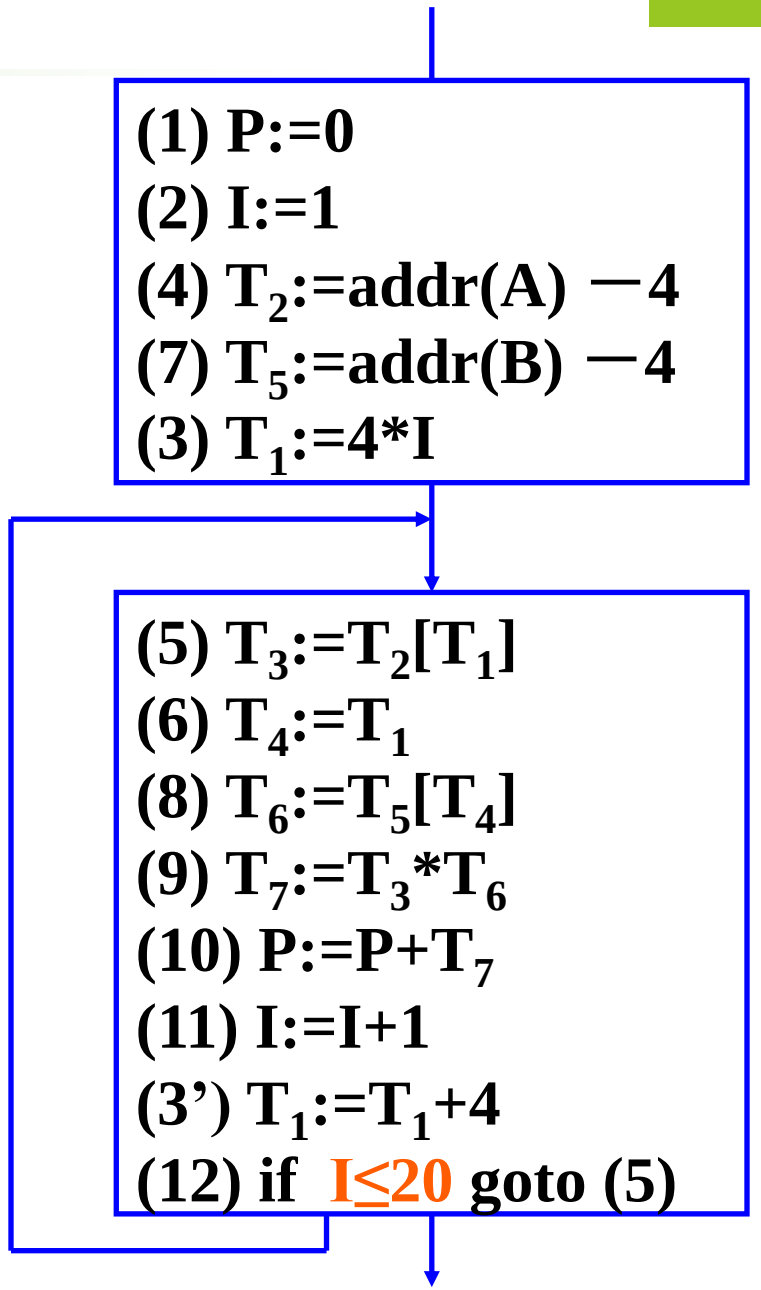
(2) 强度削弱


(1) P:=0
(2) I:=1
(4) T2:=addr(A) －4
(7) T5:=addr(B) －4
(3) T1:=4*I
(5) T3:=T2[T1]
(6) T4:=T1
(8) T6:=T5[T4]
(9) T7:=T3*T6
(10) P:=P+T7
(11) I:=I+1
(12) If I≤20 goto (3)
归纳变量消除
乘法变加法
(1) P:=0
(2) I:=1
(4) T2:=addr(A) －4
(7) T5:=addr(B) －4
(3) T1:=4*I
(5) T3:=T2[T1]
(6) T4:=T1
(8) T6:=T5[T4]
(9) T7:=T3*T6
(10) P:=P+T7
(11) I:=I+1
(3') T1:=T1+4
(12) if I≤20 goto (5)
中间代码段
强度削弱

2. 优化技术

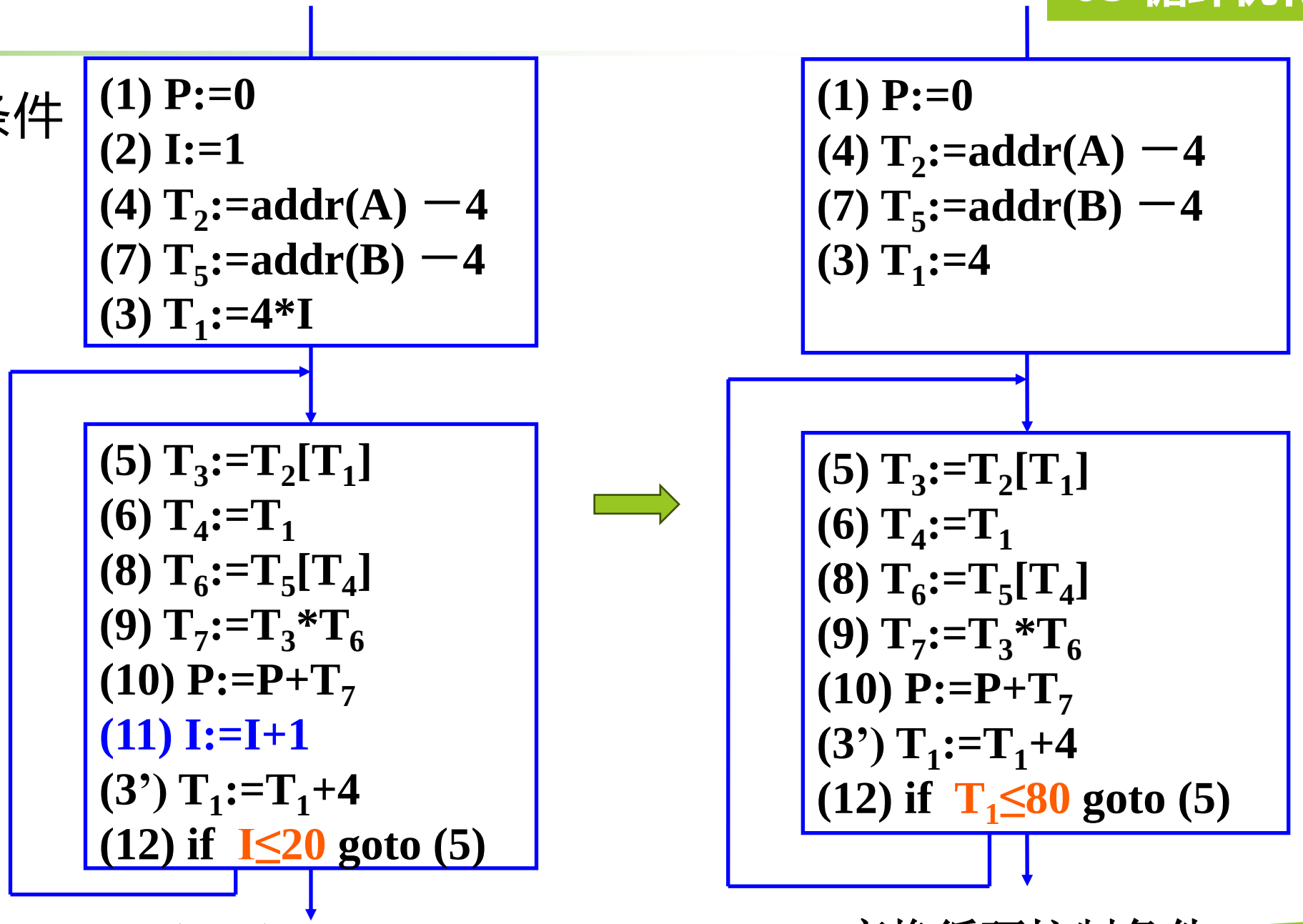
(3) 变换循环控制条件[Loop Condition Transformation]

- 将循环控制条件更换为**等价**的另一个循环控制条件
- **变换后可以删减代码或减少执行次数**
  - ✓ I和T1始终保持T1:=4*I的线性关系
  - ✓ 这样把(12)的循环控制条件 I≤20变换成T1≤80，程序的运行结果不变



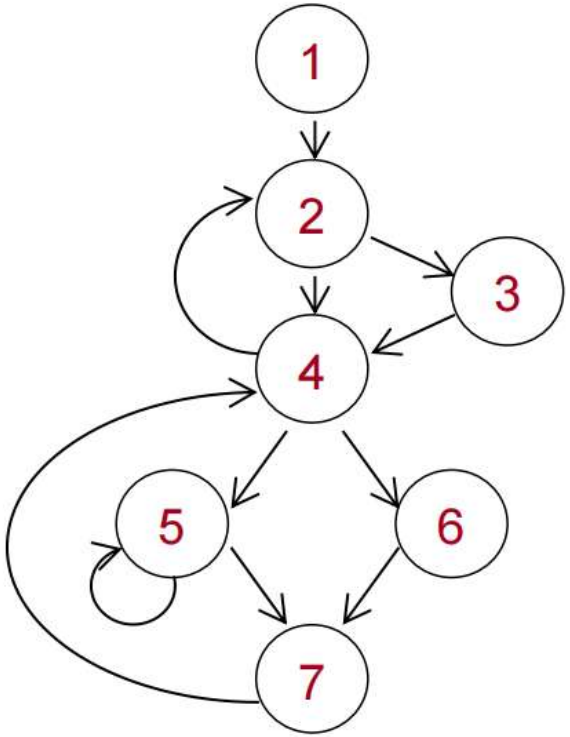
2. 优化技术

(3) 变换循环控制条件



3. 难点&重点：如何在流图中确定循环[Loop]

- 循环[也称自然循环，natural loop]
  - **强连通子图**[strongly connected subgraph]
  - 具有**唯一的入口**（头结点[header]）（**入口：外部结点进入循环的入口**）
  - 循环可以表示为一个结点序列



There are 3 loops:

{5}

{4, 5, 6, 7} 入口为4

{2, 3, 4, 5, 6, 7} 入口为2

They are NOT loops:

{2, 4} Both 2 and 4 are entries

{2, 3, 4} Both 2 and 4 are entries

{4, 5, 7} Both 4 and 7 are entries

{4, 6, 7} Both 4 and 7 are entries

3. 难点&重点：如何在流图中确定循环[Loop]

- 基本思路：根据流图计算所有结点的支配结点集[Dominators]，然后得到流图中的回边[Back Edges]，根据回边就可以确定该流图中的循环。
  - ① 计算支配结点集[Dominators]
  - ② 得到回边[Back Edges]
  - ③ 确定循环[Loop]

3. 难点&重点：如何在流图中确定循环[Loop]

① 计算支配结点集[Dominators]

– 对任意两个结点m和n，若从流图首结点出发，到达n的任一通路都要经过m，则称m是n的支配结点，记为：m **DOM** n

– 流图中结点n的所有支配结点的集合成为结点n的支配结点集，记为D(n)

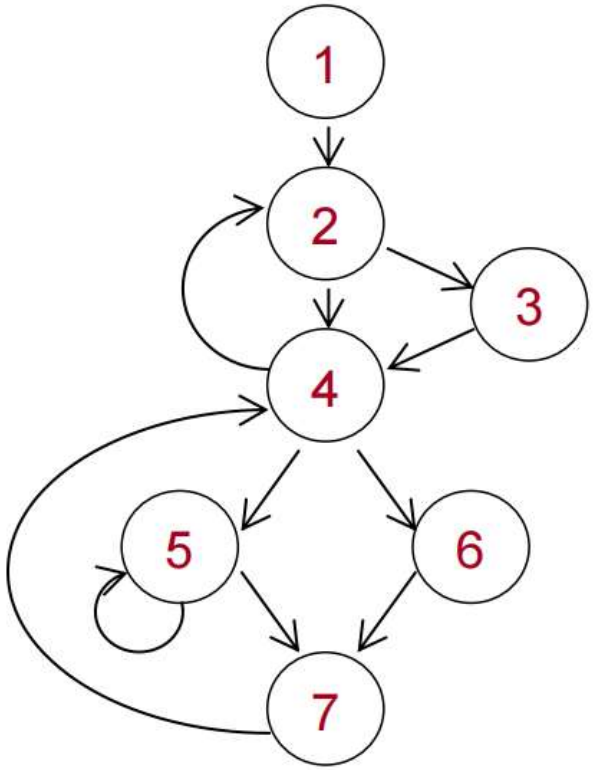
D(n) = {m | m **DOM** n}

– 设n0为流图中的首结点，

流图中任意结点a，必有

✓ a **DOM** a

✓ n0 **DOM** a



D(1) = {1}
D(2) = {1, 2}
D(3) = {1, 2, 3}
D(4) = {1, 2, 4}
D(5) = {1, 2, 4, 5}
D(6) = {1, 2, 4, 6}
D(7) = {1, 2, 4, 7}

3. 难点&重点：如何在流图中确定循环[Loop]

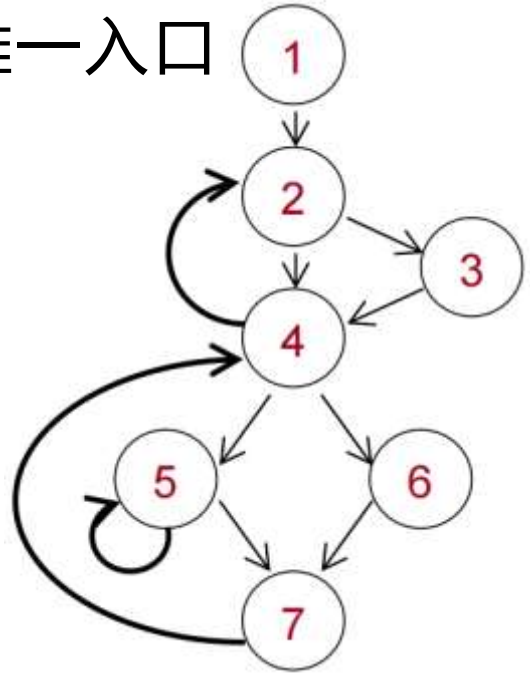
② 得到回边[Back Edges]

– 假设a→b是流图中的一条有向边，如果b DOM a，则称a→b是流图中的一条回边

③ 确定循环[Loop]

– 如果a→b是回边，则结点a，结点b，以及能到达a且不需要经过b的所有结点组成循环，且b是该循环的唯一入口

D(1) = {1}
D(2) = {1, 2}
D(3) = {1, 2, 3}
D(4) = {1, 2, 4}
D(5) = {1, 2, 4, 5}
D(6) = {1, 2, 4, 6}
D(7) = {1, 2, 4, 7}



There are 3 back edges:
5 → 5
7 → 4
4 → 2

There are 3 natural loops:
{5} defined by **5 → 5**
{4, 5, 6, 7} defined by **7 → 4**
{2, 3, 4, 5, 6, 7} defined by **4 → 2**

3. 难点&重点：如何在流图中确定循环[Loop]

③ 确定循环[Loop]

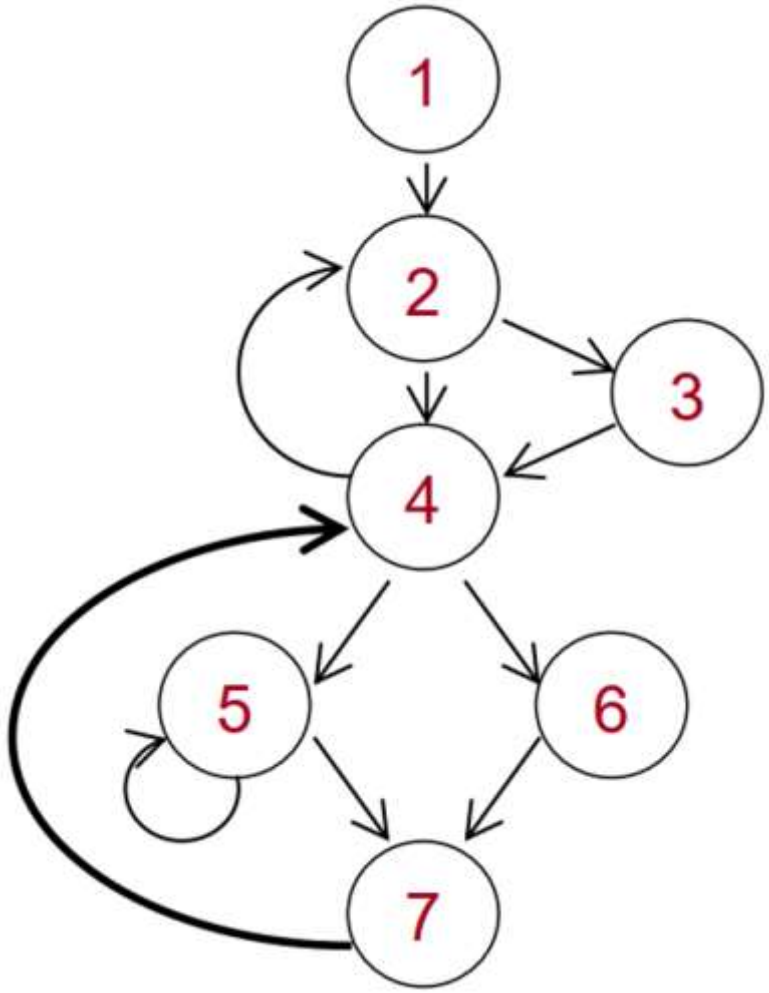
Input: back edge $n \rightarrow d$.

Algorithm:

```
1. void insert(Node m) {
2.    if (m ∉ loop) {  // d will not be pushed
3.      loop = loop ⋃ {m};
4.      stack.push(m)
5.    }
6. }
7. void main() {
8.    Stack stack = new Stack();
9.    loop = {d};  // PRE(d) will not be added
10.   insert(n);
11.   while (stack.notEmpty()) {
12.     m = stack.pop();
13.     foreach (p ∈ PRE(m))  insert(p)
14.   }
15.}
```

3. 难点&重点：如何在流图中确定循环[Loop]

③ 确定循环[Loop]



Given the back edge $7 \rightarrow 4$

1. initialize: loop = {4, 7}, stack = [7].
2. pop 7; insert 5 and 6;

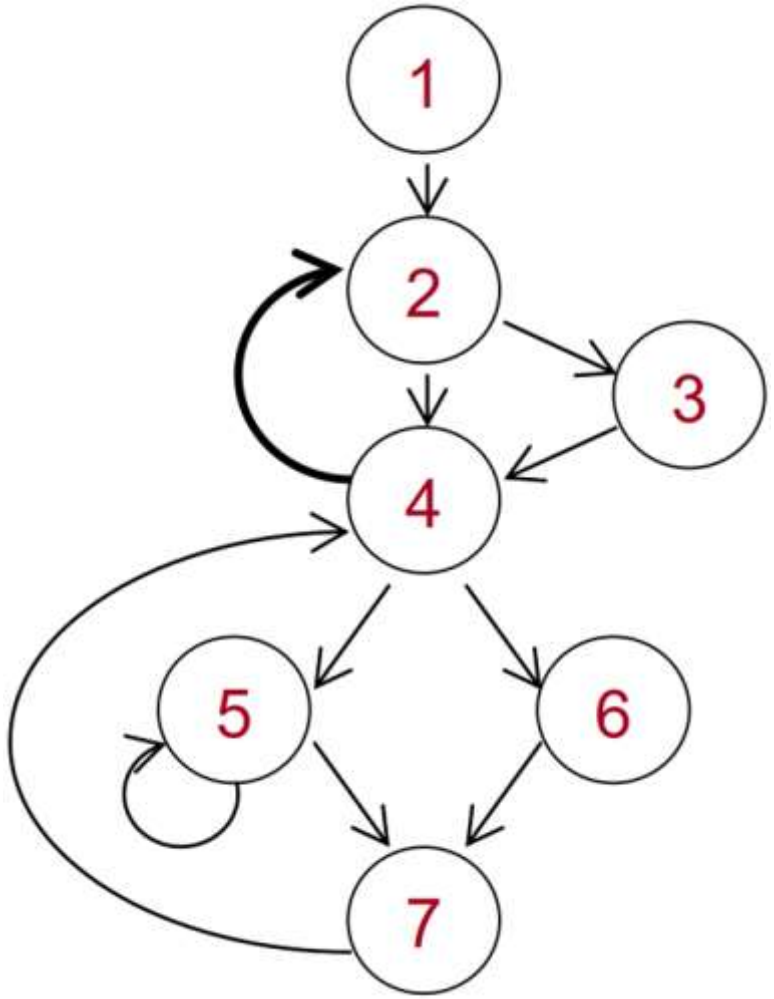
   loop = {4, 7, 5, 6}, stack = [5, 6].
3. pop 6; insert 4 (already in loop);

   loop has no change, stack = [5].
4. pop 5; insert 4 and 5 (both in loop);

   loop has no change, stack = [].
5. result: loop = {4, 7, 5, 6}.

3. 难点&重点：如何在流图中确定循环[Loop]

③ 确定循环[Loop]



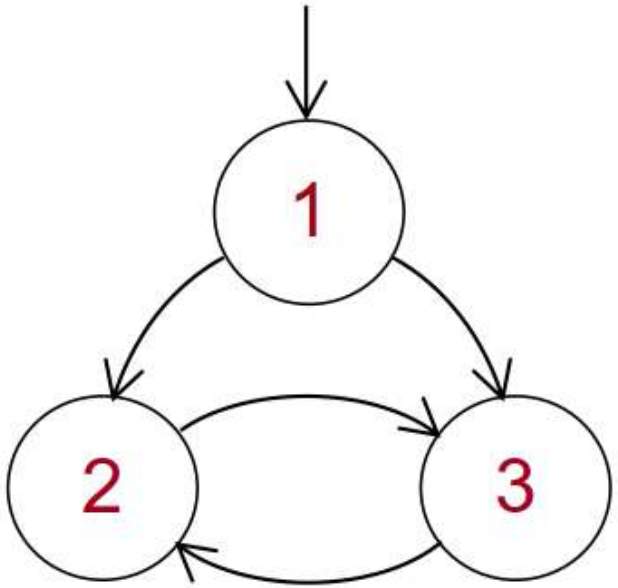
Given the back edge 4 → 2

1. initialize: loop = {2, 4}, stack = [4].
2. pop 4; insert 2, 3 and 7 (2 already in loop);
   loop = {2, 4, 3, 7}, stack = [3, 7].
3. pop 7; insert 5 and 6;
   loop = {2, 4, 3, 7, 5, 6}, stack = [3, 5, 6].
4. pop 6; insert 4 (already in loop);
   loop had no change, stack = [3, 5].
5. pop 5; insert 4 and 5 (both in loop);
   loop has no change, stack = [3].
6. pop 3; insert 2 (already in loop);
   loop has no change, stack = [].
7. result: loop = {2, 4, 3, 7, 5, 6}.

4. 自然循环的性质

- 自然循环并不能涵盖常识中的所有循环
  - 例如，在下面的流图中不存在回边，但按照常识它确实有一个循环：{2, 3}。
  - 只有在可归约流图中，回边才能找到所有循环。

- 可归约流图[Reducible flow graph]
  - 移除所有回边后，子图是无环的。
  - 在可归约流图中，循环的唯一入口是头节点。
  - 由结构化程序生成的流图通常是可归约的。



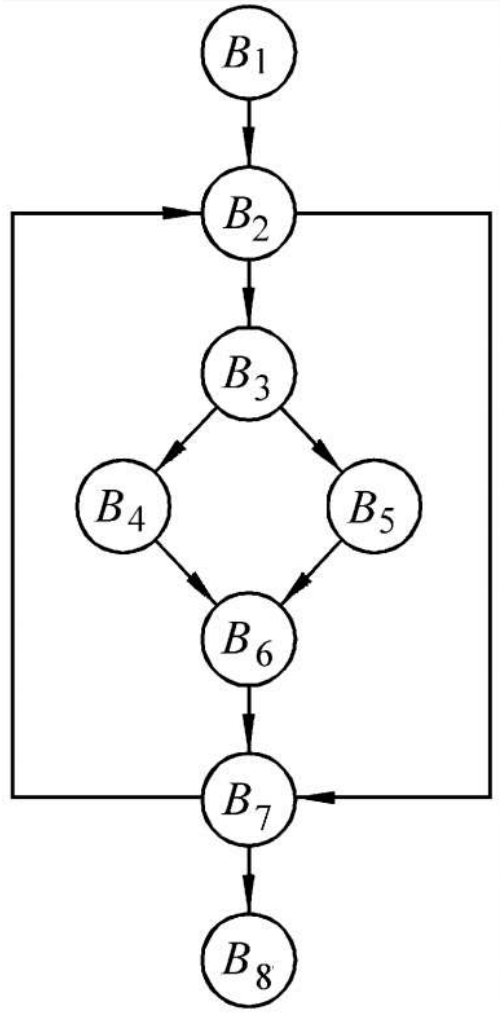
随堂练习（2）

对于右侧流图：

(1) 求出流图中各节点n的支配结点集D(n)；

(2) 求出流图中的回边；

(3) 求出流图中的循环。



(1) 支配结点集：

$D(B_1)=\{B_1\}$
$D(B_2)=\{B_1, B_2\}$
$D(B_3)=\{B_1, B_2, B_3\}$
$D(B_4)=\{B_1, B_2, B_3, B_4\}$
$D(B_5)=\{B_1, B_2, B_3, B_5\}$
$D(B_6)=\{B_1, B_2, B_3, B_6\}$
$D(B_7)=\{B_1, B_2, B_7\}$
$D(B_8)=\{B_1, B_2, B_7, B_8\}$

(2) 回边：(B7→B2)

(3) 循环：对应回边(B7→B2)，循环为：B2,B3,B4,B5,B6,B7

CONTENTS

目 录

01 代码优化简介 Introduction

02 局部优化 Local Optimization

03 循环优化 Loop Optimization

04 全局优化 Global Optimization

1. 概述

- 全局优化作用于**整个过程（函数），**可以进行**跨基本块**的优化
- 常用技术：
  - 全局公共子表达式删除
  - 常量折叠和传播
  - 死代码删除
  - 复写传播
- **关键在于数据流分析**
  - 检测变量、赋值、寄存器分配等

2. 数据流分析[Data-flow Analysis]

- 控制流 vs. 数据流
  - 控制流：将基本块视为black boxes
  - **数据流**：将基本块视为**white boxes**
- 收集关于数据流的信息
  - 变量如何**被赋值（定义）**，左值[L-value]
  - 变量如何**被引用（使用）**，右值[R-value]

2. 数据流分析[Data-flow Analysis]

- 何时需要全局信息?
  - 局部优化
    - ✓ 如果变量从未被使用，则可以删除对该变量的赋值
  - 循环优化：代码外提
    - ✓ 根据变量的定义，确定循环不变运算
    - ✓ 代码外提要求该运算是循环内唯一的定义
    - ✓ 代码外提还要求所定义的变量在循环出口处不是活跃的（not live）
  - 循环优化：归纳变量消除
    - ✓ 如果归纳变量在循环外未被使用，则可以将其消除
  - 代码生成
    - ✓ 出口处的活跃性信息有助于提高寄存器利用率

2. 数据流分析[Data-flow Analysis]

- 需要哪些全局信息?
  - **定义[Definition]**
    - ✓ 语句中对某个左值[L-value]的所有赋值（来源）
  - **使用[Use]**
    - ✓ 语句中对某个右值[R-value]的所有可能使用
  - **活跃性[Liveness]**
    - ✓ 变量在某个语句之后是否会被作为右值[R-value]引用

2. 数据流分析[Data-flow Analysis]

- 两类数据流分析
  - **到达-定值**数据流分析[Reaching Definition Analysis]
  - **活跃变量**数据流分析[Live Variable Analysis]

2. 数据流分析[Data-flow Analysis]

- **到达-定值**数据流分析[Reaching Definition Analysis]
  - 向前流：信息流的方向与控制流是一致的
  - 变量A的**定值[definition]**是对A的赋值或读值到A的语句，该语句的位置称作A的**定值点**
  - 变量A的**定值点**d**到达**某点p，是指流图中从d有一条路径到达p且该通路上没有A的其它定值
    - ✓ 当我们在p之后立即使用变量A时，A的值可能由d决定

2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
    - ✓ **OUT[B]为所有该基本块B入口处的定值点集合IN[B]去除当前基本块“杀死”的定值点，再加入当前基本块新产生的定值点**
  - IN[B]=∪OUT[b]　b∈P[B]
    - ✓ **IN[B]为B的所有前驱基本块的出口处OUT集合的并集**
  - 其中:
    - B：基本块；P[B]：B的所有前驱基本块；
    - IN[B]：到达B入口处的各个变量的所有定值点集合；
    - OUT[B]：到达B出口处的各个变量的所有定值点集合；
    - GEN[B]：B中定值并可到达B出口处的所有定值点集合；
    - KILL[B]：B之外的能够到达B的入口处、且其定值的变量在B中又重新定值（即被B所“杀死”）的定值点集合。

2. 数据流分析[Data-flow Analysis]

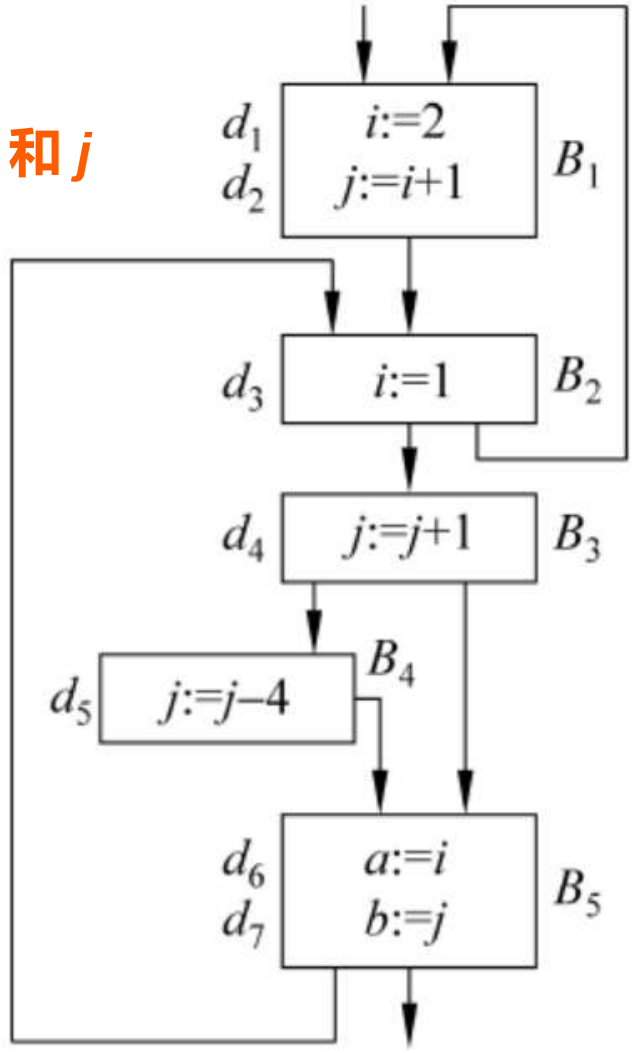
- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*

初始状态

	GEN	KILL	IN	OUT
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	{ }	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	{ }	$\{d_3\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	{ }	$\{d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	{ }	$\{d_5\}$
B_5	{ }	{ }	{ }	{ }

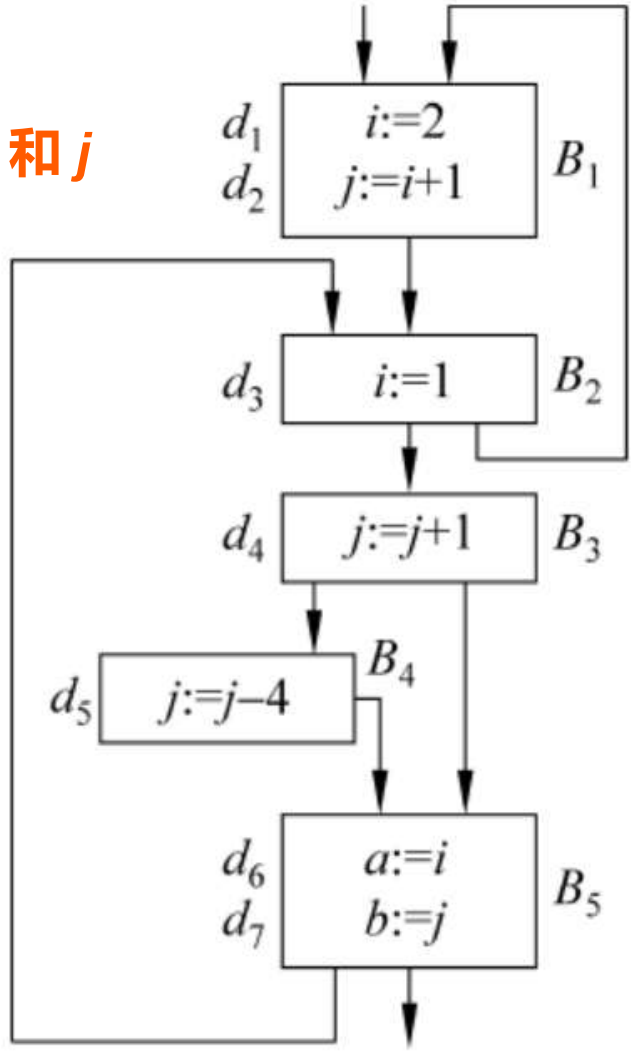
向量表示：1100000



2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*



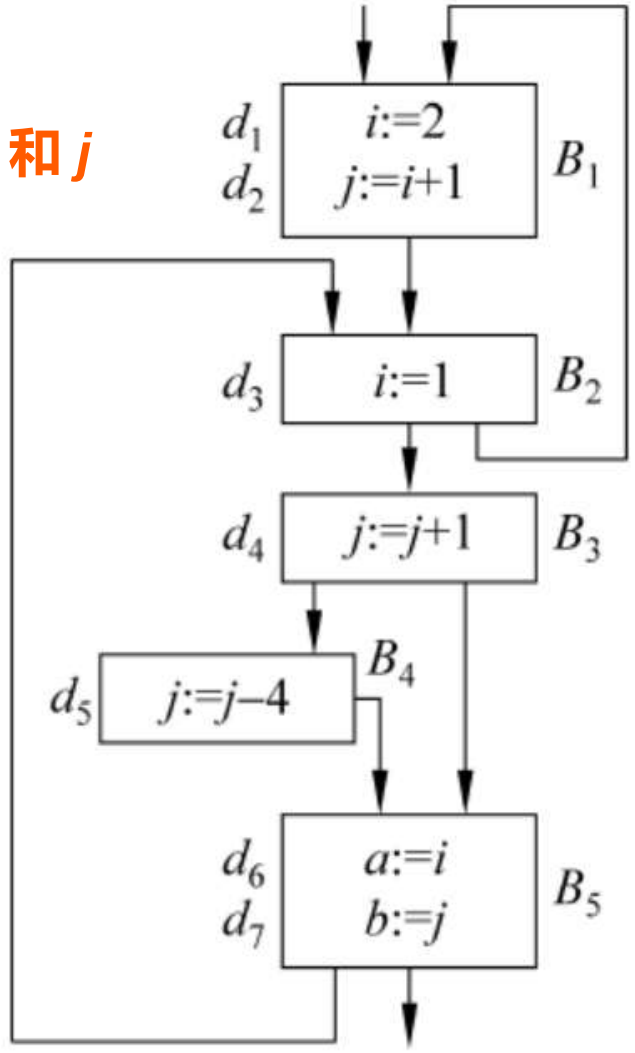
第1次迭代

	GEN	*KILL*	*IN*	*OUT*
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	$\{d_3\}$	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	$\{\ \}$	$\{d_3\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	$\{\ \}$	$\{d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	$\{\ \}$	$\{d_5\}$
B_5	$\{\ \}$	$\{\ \}$	$\{\ \}$	$\{\ \}$

2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*



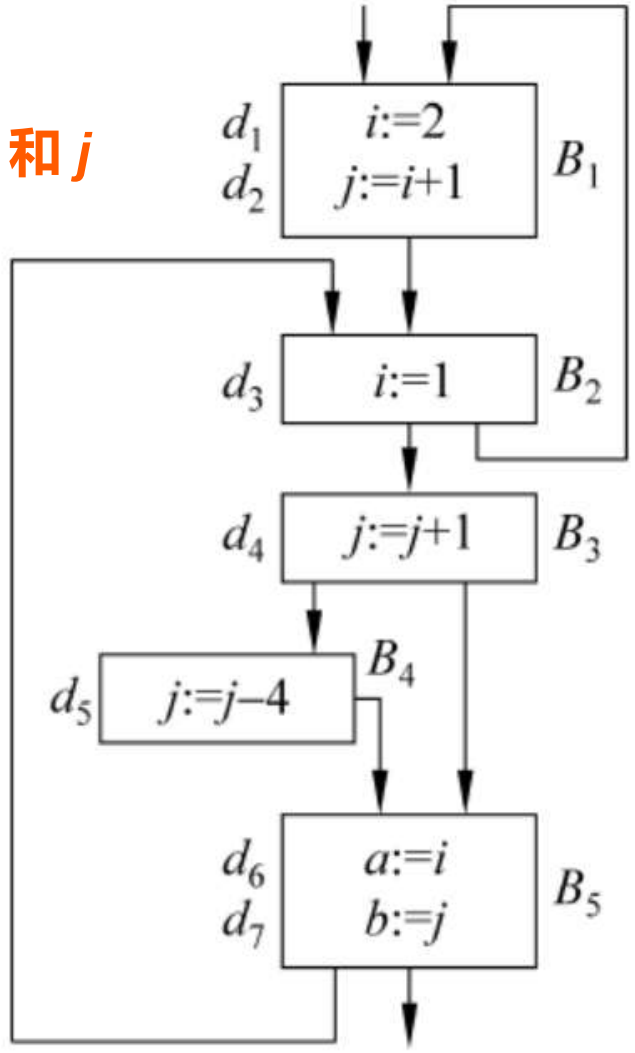
第**1**次迭代

	GEN	*KILL*	*IN*	*OUT*
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	$\{d_3\}$	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	$\{d_1,d_2\}$	$\{d_2,d_3\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	$\{\ \}$	$\{d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	$\{\ \}$	$\{d_5\}$
B_5	$\{\ \}$	$\{\ \}$	$\{\ \}$	$\{\ \}$

2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*



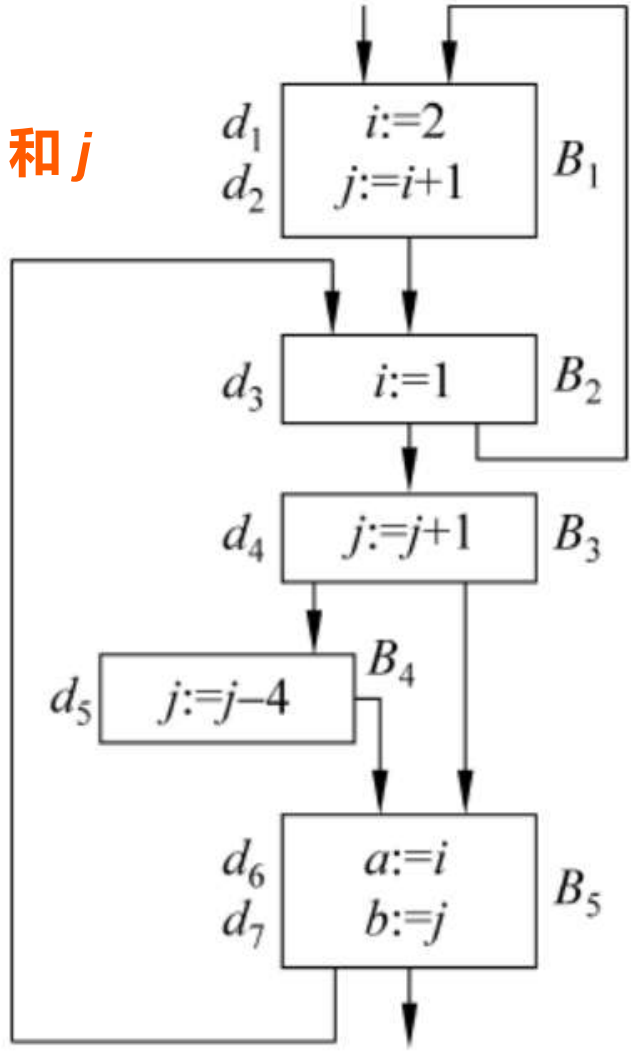
第1次迭代

	GEN	*KILL*	*IN*	*OUT*
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	$\{d_3\}$	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	$\{d_1,d_2\}$	$\{d_2, d_3\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	$\{d_2, d_3\}$	$\{d_3,d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	$\{\ \}$	$\{d_5\}$
B_5	$\{\ \}$	$\{\ \}$	$\{\ \}$	$\{\ \}$

2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*



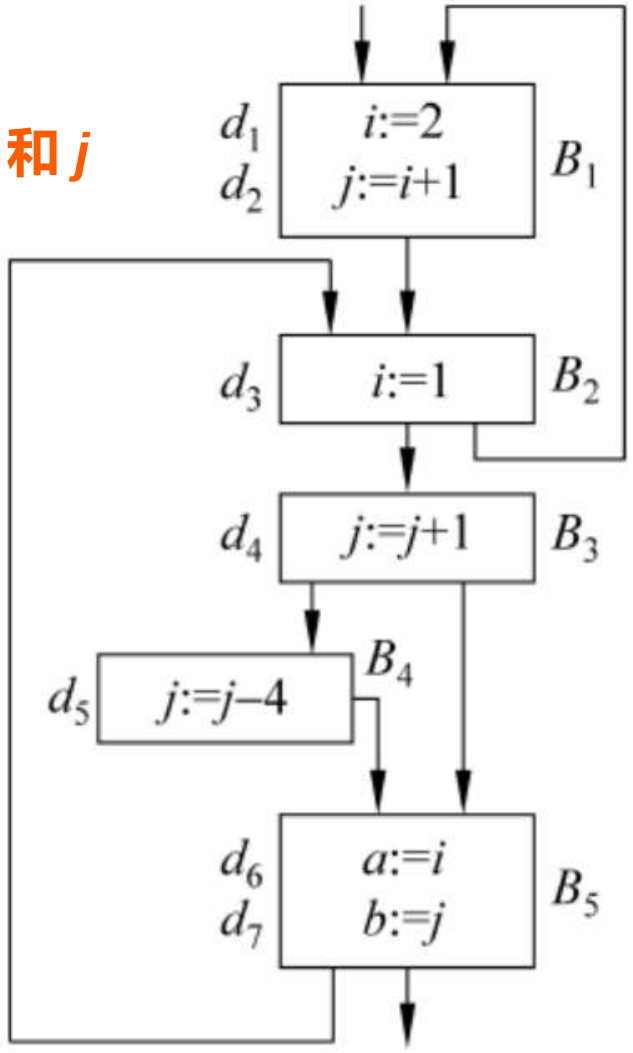
第1次迭代

	GEN	*KILL*	*IN*	*OUT*
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	$\{d_3\}$	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	$\{d_1,d_2\}$	$\{d_2, d_3\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	$\{d_2, d_3\}$	$\{d_3,d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	$\{d_3,d_4\}$	$\{d_3,d_5\}$
B_5	{ }	{ }	{ }	{ }

2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*



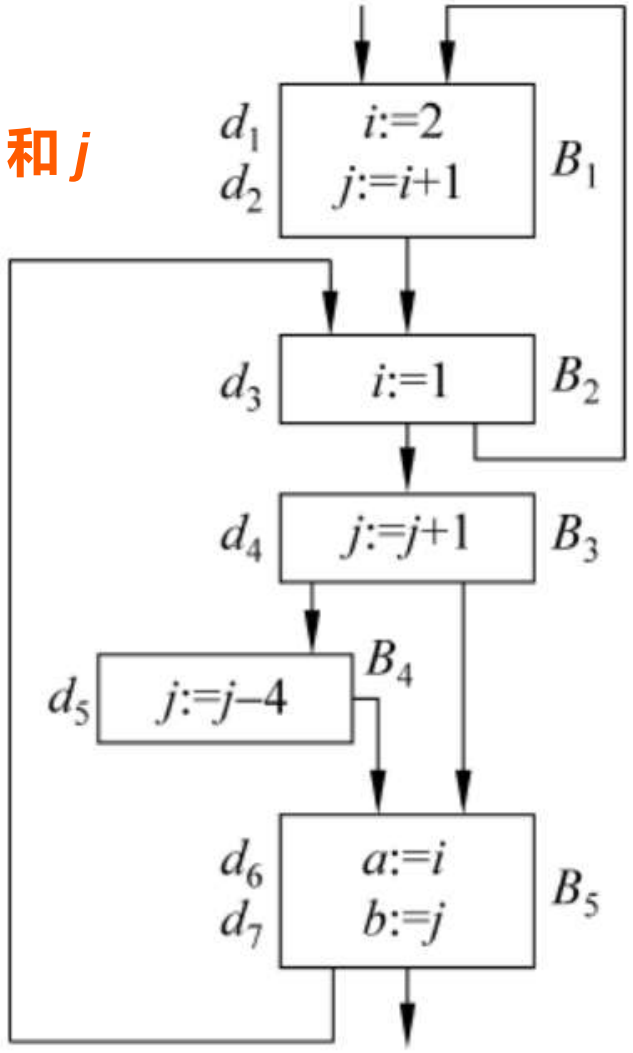
第1次迭代

	GEN	*KILL*	*IN*	*OUT*
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	$\{d_3\}$	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	$\{d_1,d_2\}$	$\{d_2, d_3\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	$\{d_2, d_3\}$	$\{d_3,d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	$\{d_3,d_4\}$	$\{d_3,d_5\}$
B_5	$\{\ \}$	$\{\ \}$	$\{d_3,d_4,d_5\}$	$\{d_3,d_4,d_5\}$

2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*



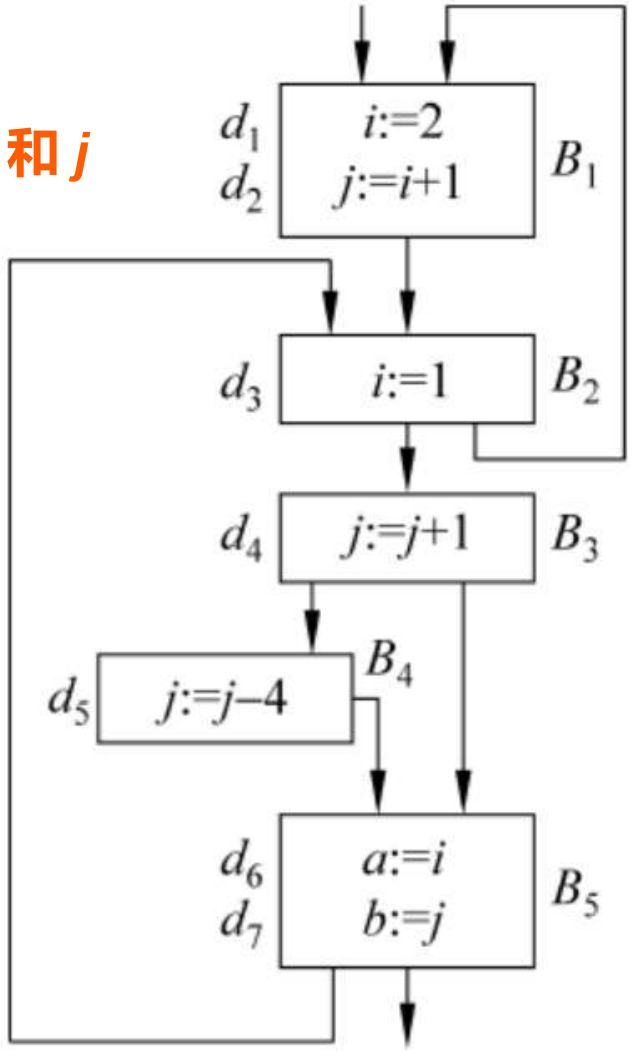
第**2**次迭代

	GEN	*KILL*	*IN*	*OUT*
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	$\{d_2, d_3\}$	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	$\{d_1,d_2\}$	$\{d_2, d_3\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	$\{d_2, d_3\}$	$\{d_3,d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	$\{d_3,d_4\}$	$\{d_3,d_5\}$
B_5	$\{\ \}$	$\{\ \}$	$\{d_3,d_4,d_5\}$	$\{d_3,d_4,d_5\}$

2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*



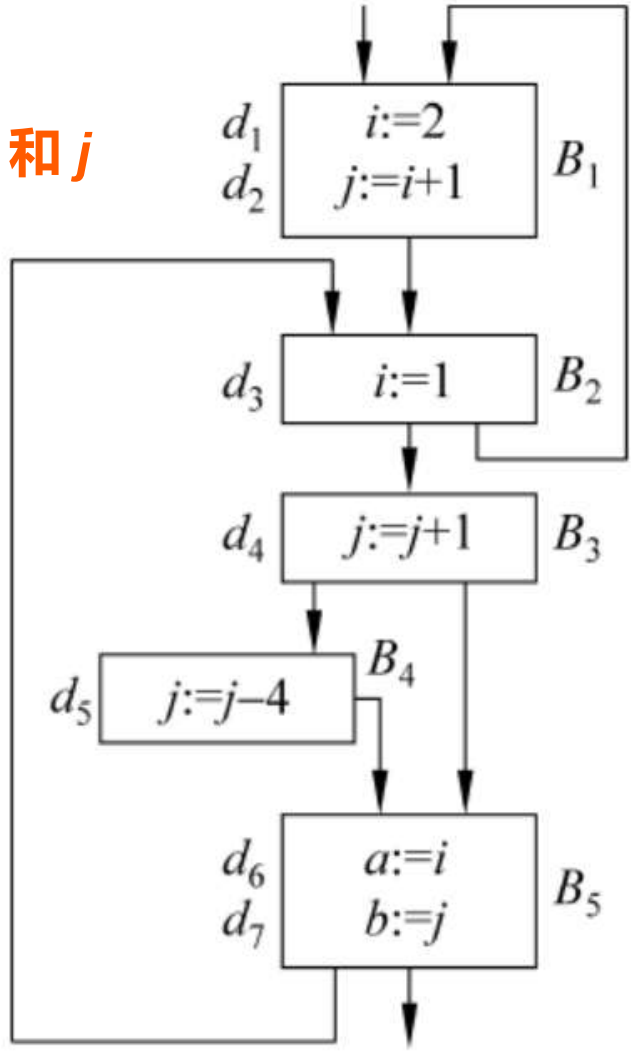
第2次迭代

	GEN	*KILL*	*IN*	*OUT*
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	$\{d_2, d_3\}$	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	$\{d_1, d_2, d_3,d_4,d_5\}$	$\{d_2, d_3,d_4,d_5\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	$\{d_2, d_3\}$	$\{d_3,d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	$\{d_3,d_4\}$	$\{d_3,d_5\}$
B_5	{ }	{ }	$\{d_3,d_4,d_5\}$	$\{d_3,d_4,d_5\}$

2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*



第2次迭代

	GEN	*KILL*	*IN*	*OUT*
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	$\{d_2, d_3\}$	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	$\{d_1, d_2, d_3,d_4,d_5\}$	$\{d_2, d_3,d_4,d_5\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	$\{d_2, d_3,d_4,d_5\}$	$\{d_3,d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	$\{d_3,d_4\}$	$\{d_3,d_5\}$
B_5	{ }	{ }	$\{d_3,d_4,d_5\}$	$\{d_3,d_4,d_5\}$

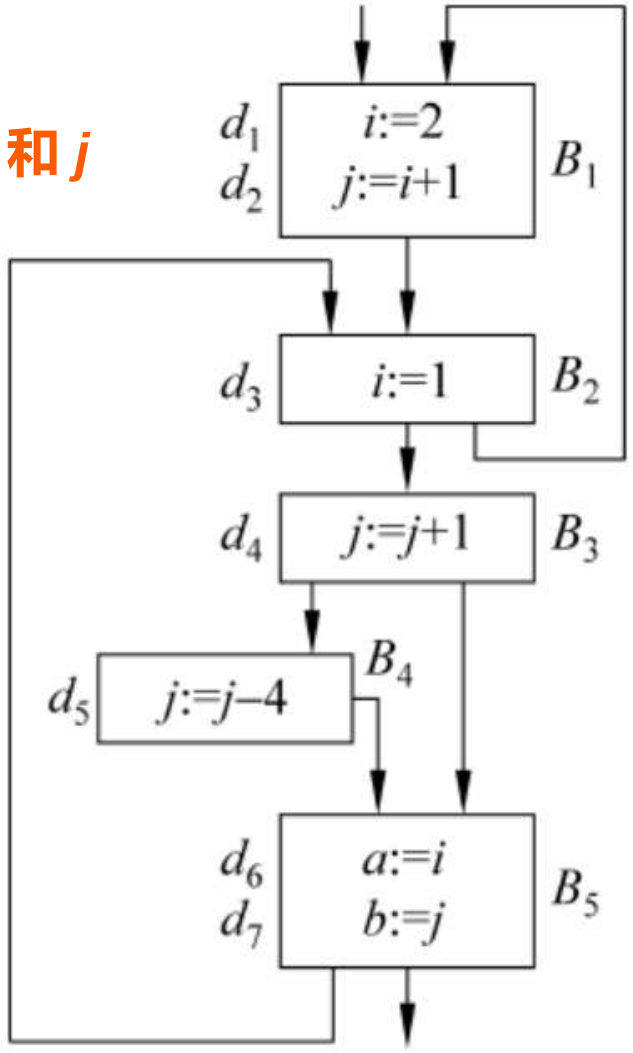
2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - OUT[B]=GEN[B]∪(IN[B]–KILL[B])
  - IN[B]=∪OUT[b]　b∈P[B]

假设仅考虑变量 *i* 和 *j*

第**3**次迭代

	GEN	*KILL*	*IN*	*OUT*
B_1	$\{d_1,d_2\}$	$\{d_3,d_4,d_5\}$	$\{d_2, d_3,d_4,d_5\}$	$\{d_1,d_2\}$
B_2	$\{d_3\}$	$\{d_1\}$	$\{d_1, d_2, d_3,d_4,d_5\}$	$\{d_2, d_3,d_4,d_5\}$
B_3	$\{d_4\}$	$\{d_2,d_5\}$	$\{d_2, d_3,d_4,d_5\}$	$\{d_3,d_4\}$
B_4	$\{d_5\}$	$\{d_4\}$	$\{d_3,d_4\}$	$\{d_3,d_5\}$
B_5	{ }	{ }	$\{d_3,d_4,d_5\}$	$\{d_3,d_4,d_5\}$



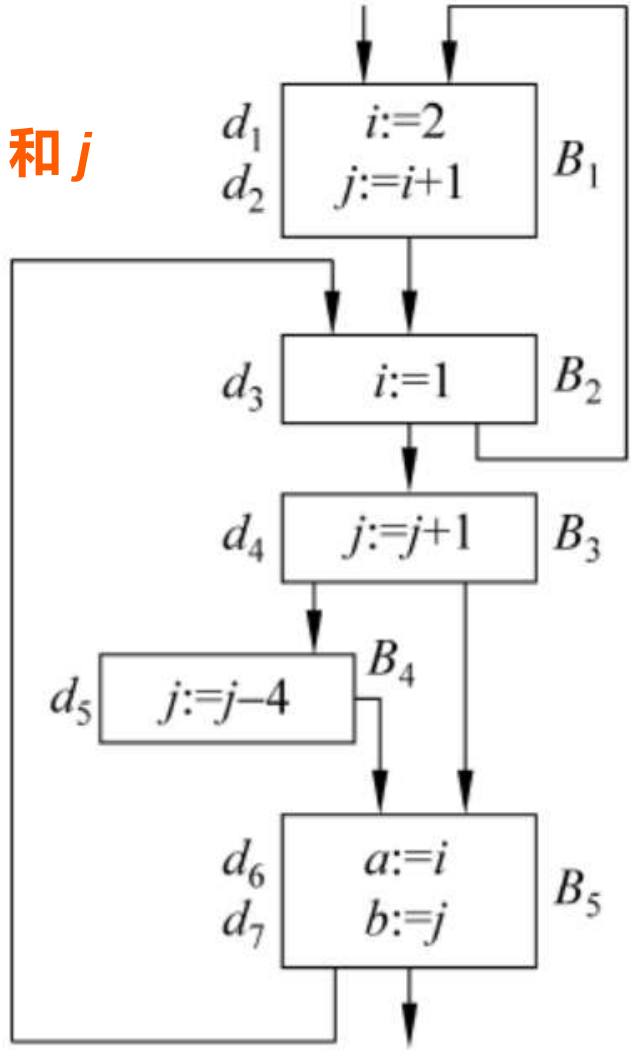
2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - **UD链**[Use-Definition Chaining, 引用-定值链]：变量A在引用语句u的定值链
    - ✓ **能够到达语句u的所有变量A定值的集合**
    - ✓ 可用于查找循环不变量，也可用于全局常量折叠优化
    - ✓ 定义：设在程序中某点u引用了变量A的值，则把能到达u的A的所有定值点的全体，称为A在引用点u的引用-定值链，简称UD链。
    - ✓ 如何计算：
      - 如果在基本块B中，变量 A 的引用点u之前有A的定值点d，并且A在点d的定值到达u，那么A在点u的UD链就是 {d} ；
      - 如果在基本块B中，变量A的引用点u之前没有A的定值点，那么，IN[B]中A的所有定值点均到达u，它们就是A在点u的UD链。

2. 数据流分析[Data-flow Analysis]

- 到达-定值数据流分析[Reaching Definition Analysis]
  - **UD链**

假设仅考虑变量 *i* 和 *j*



i 在引用点 d_2 的 UD 链为 $\{d_1\}$

i 在引用点 d_6 的 UD 链为 $\{d_3\}$

j 在引用点 d_4 的 UD 链为 $\{d_2, d_4, d_5\}$

j 在引用点 d_5 的 UD 链为 $\{d_4\}$

j 在引用点 d_7 的 UD链为 $\{d_4, d_5\}$

2. 数据流分析[Data-flow Analysis]

- **活跃变量**数据流分析[Live Variable Analysis]
  - 向后流：信息流的方向与控制流反向
  - 活跃变量
    - ✓ 对程序中的某变量A和某点p而言，如果存在一条从p开始的通路，其中引用了A在点p的值，则称A在点p是**活跃的**
    - ✓ 直观地，对于全局范围的分析来说，**一个变量是活跃的，如果存在一条路径使得该变量被重新定值之前它的当前值还要被引用**

2. 数据流分析[Data-flow Analysis]

- 活跃变量数据流分析[Live Variable Analysis]
  - LiveIn(B)= LiveUse(B)∪(LiveOut(B)-Def(B))
    - **变量在B中定值前有引用，或者在B出口处活跃且未在B中被定值，则它在B入口处是活跃的**
  - LiveOut(B) =∪LiveIn(s) $s \in S[B]$
    - **变量在B出口处活跃，当且仅当它在B的某个后继基本块入口处活跃**
  - 其中：
    - B为一个基本块，S[B]为B的所有后继基本块；
    - LiveUse(B)为B中被定值之前要引用变量的集合；
    - Def(B)为在B中定值的且定值前未曾在B中引用过的变量集合；
    - LiveIn(B)为B入口处为活跃的变量的集合；
    - LiveOut(B)为B出口处为活跃的变量的集合。

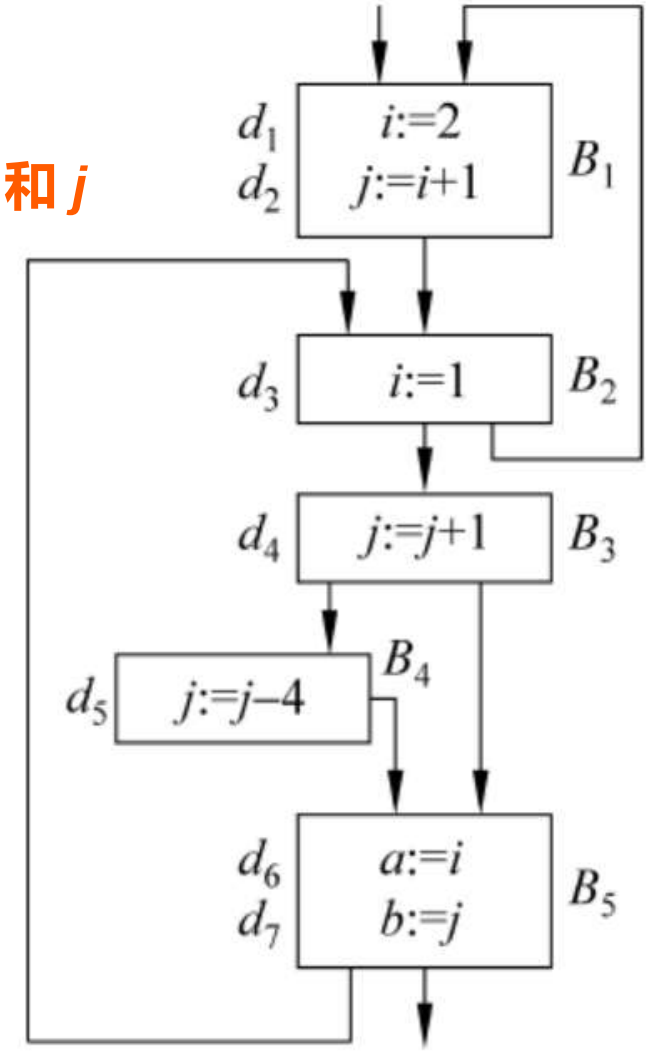
2. 数据流分析[Data-flow Analysis]

- 活跃变量数据流分析[Live Variable Analysis]
  - LiveIn(B)= LiveUse(B)∪(LiveOut(B)-Def(B))
  - LiveOut(B) =∪LiveIn(s)　$s \in S[B]$

初始化

	Def	LiveUse
B_1	{i,j}	Ø
B_2	{i}	Ø
B_3	Ø	{j}
B_4	Ø	{j}
B_5	Ø	{i,j}

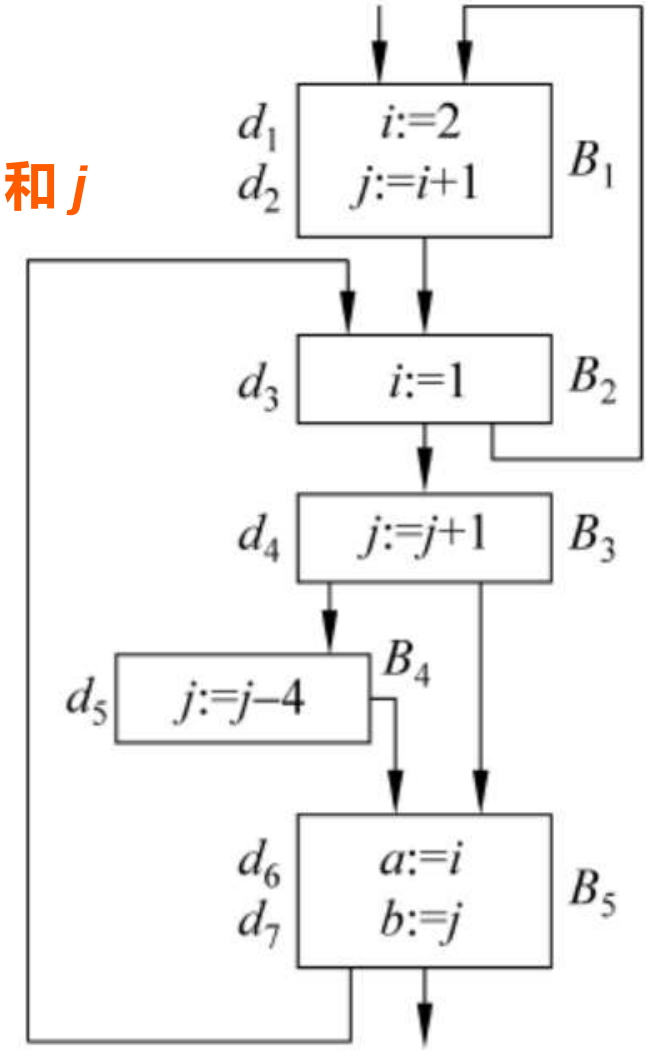
假设仅考虑变量 *i* 和 *j*



2. 数据流分析[Data-flow Analysis]

- 活跃变量数据流分析[Live Variable Analysis]
  - LiveIn(B)= LiveUse(B)∪(LiveOut(B)-Def(B))
  - LiveOut(B) =∪LiveIn(s)　$s \in S[B]$

假设仅考虑变量 *i* 和 *j*



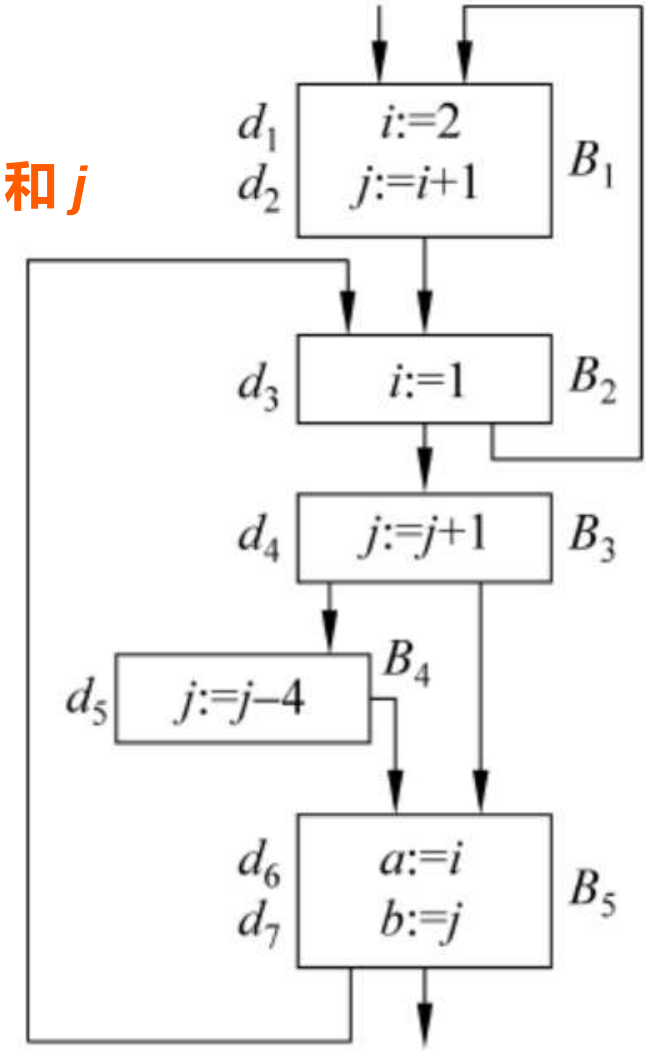
第1次迭代

	Def	LiveUse	LiveOut	LiveIn
B_1	{i,j}	∅	∅	∅
B_2	{i}	∅	∅	∅
B_3	∅	{j}	∅	{j}
B_4	∅	{j}	∅	{j}
B_5	∅	{i,j}	∅	{i,j}

2. 数据流分析[Data-flow Analysis]

- 活跃变量数据流分析[Live Variable Analysis]
  - LiveIn(B)= LiveUse(B)∪(LiveOut(B)-Def(B))
  - LiveOut(B) =∪LiveIn(s)　$s \in S[B]$

假设仅考虑变量 *i* 和 *j*



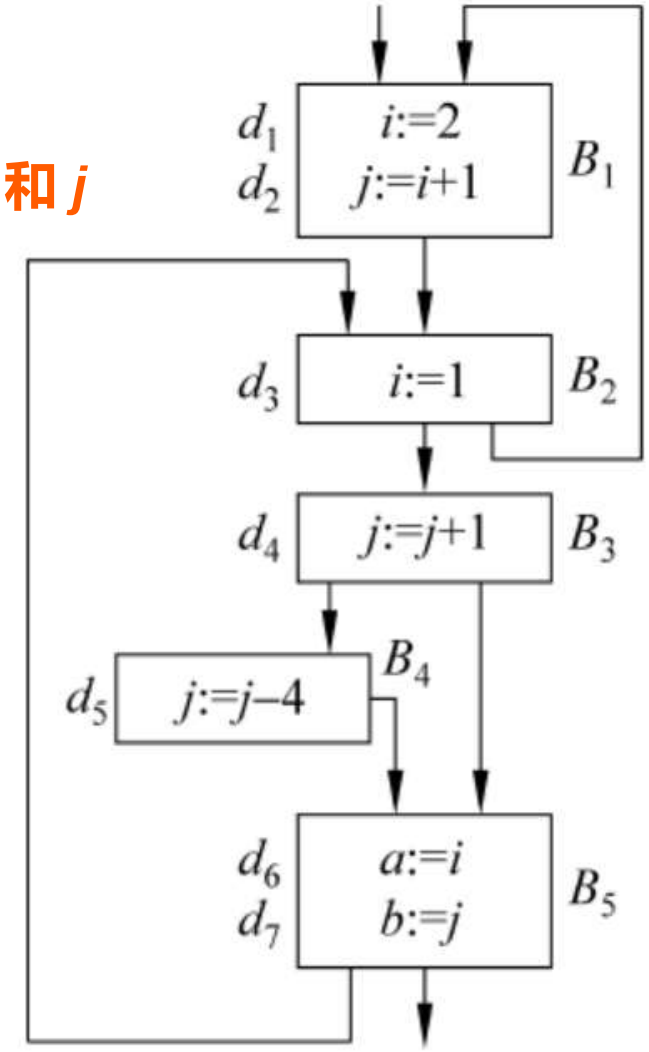
第2次迭代

	Def	LiveUse	LiveOut	LiveIn
B_1	{i,j}	∅	∅	∅
B_2	{i}	∅	{j}	∅
B_3	∅	{j}	{i,j}	{j}
B_4	∅	{j}	{i,j}	{j}
B_5	∅	{i,j}	∅	{i,j}

2. 数据流分析[Data-flow Analysis]

- 活跃变量数据流分析[Live Variable Analysis]
  - LiveIn(B)= LiveUse(B)∪(LiveOut(B)-Def(B))
  - LiveOut(B) =∪LiveIn(s)　$s \in S[B]$

假设仅考虑变量 *i* 和 *j*



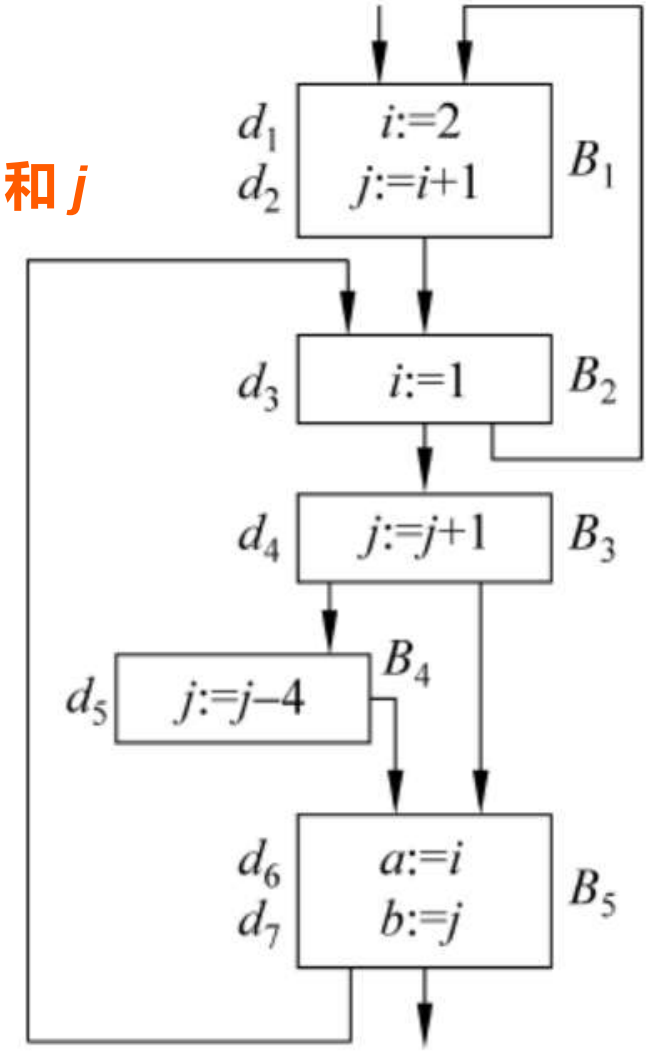
第3次迭代

	Def	LiveUse	LiveOut	LiveIn
B_1	{i,j}	∅	∅	∅
B_2	{i}	∅	{j}	{j}
B_3	∅	{j}	{i,j}	{i,j}
B_4	∅	{j}	{i,j}	{i,j}
B_5	∅	{i,j}	∅	{i,j}

2. 数据流分析[Data-flow Analysis]

- 活跃变量数据流分析[Live Variable Analysis]
  - LiveIn(B)= LiveUse(B)∪(LiveOut(B)-Def(B))
  - LiveOut(B) =∪LiveIn(s)　$s \in S[B]$

假设仅考虑变量 *i* 和 *j*



第4次迭代

	Def	LiveUse	LiveOut	LiveIn
B_1	{i,j}	∅	{j}	∅
B_2	{i}	∅	{i,j}	{j}
B_3	∅	{j}	{i,j}	{i,j}
B_4	∅	{j}	{i,j}	{i,j}
B_5	∅	{i,j}	{j}	{i,j}

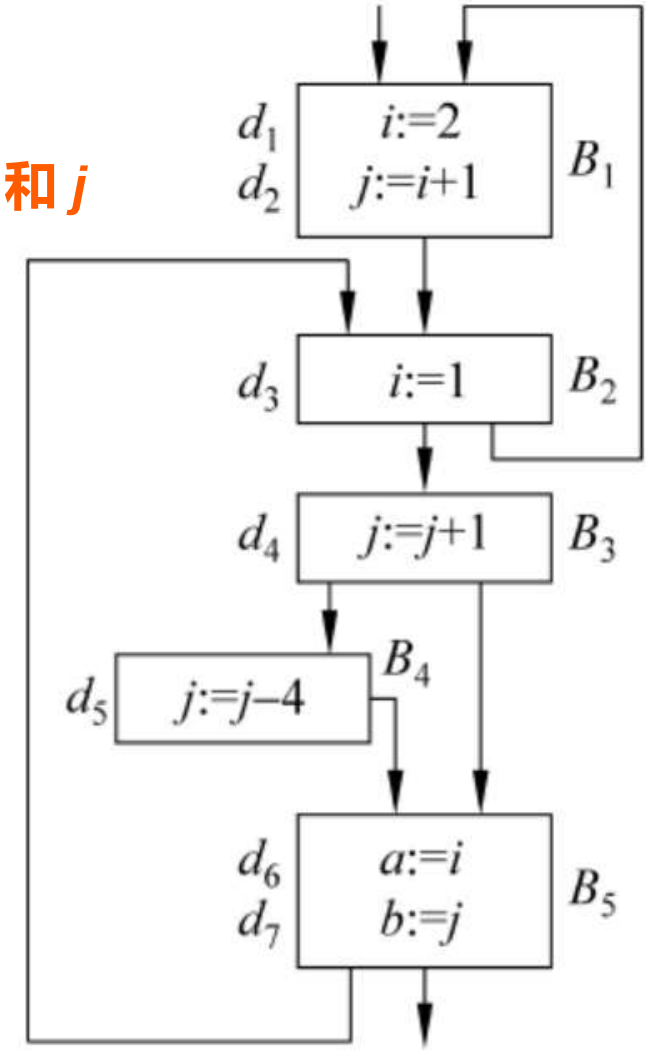
2. 数据流分析[Data-flow Analysis]

- 活跃变量数据流分析[Live Variable Analysis]
  - **DU链**[Definition-Use Chaining, 定值-引用链]：变量A在定值语句u的使用链
    - ✓ **能够从u到达的所有变量A使用的集合**
    - ✓ 定义：假设在程序中某点u定义了变量A的值，从u存在一条到达A的某个引用点s的路径，且该路径上不存在A的其他定值点，则把所有此类引用点s的全体称为A在定值点u的定值-引用链，简称DU链。
    - ✓ 计算：DU链的计算可采用类似活跃变量数据流的向后流方法

2. 数据流分析[Data-flow Analysis]

- 活跃变量数据流分析[Live Variable Analysis]
  - **DU链**[Definition-Use Chaining, 定值-引用链]

假设仅考虑变量 *i* 和 *j*



i 在定值点 d1 的 DU 链为 {d2}

j 在定值点d2 的 DU 链为 {d4}

i 在定值点 d3 的 DU 链为 {d6}

j 在定值点 d4 的 DU 链为 {d4, d5, d7}

j 在定值点 d5 的 DU 链为 {d4, d7}

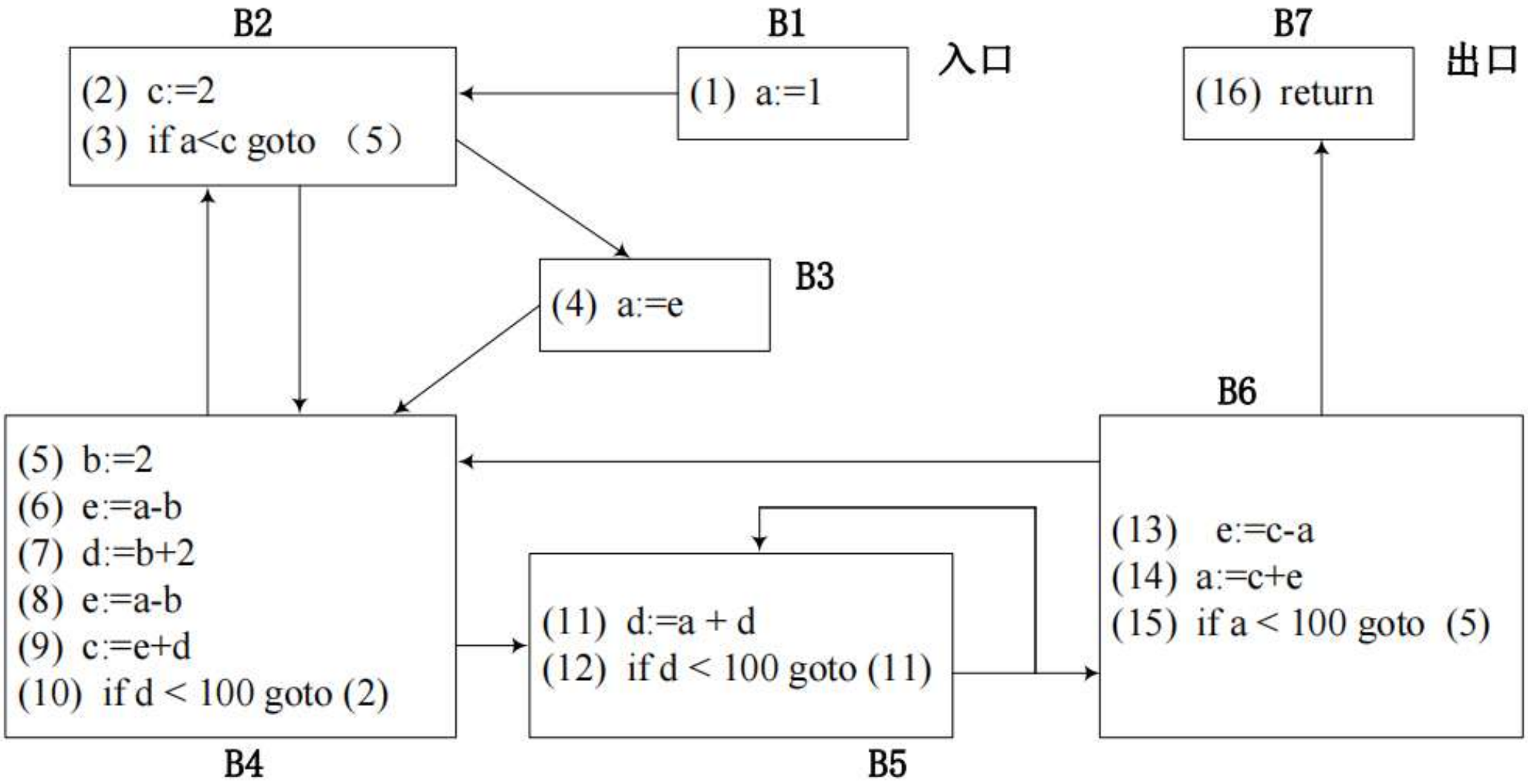
本章小结

- 重点
  - 局部优化：**为基本块构造DAG；根据DAG图优化基本块**
  - 循环优化：**如何在流图中确定循环；基本循环优化技术**
  - 全局优化：**到达-定值数据流分析/UD链；活跃变量数据流分析/DU链**

- 难点
  - 循环优化：如何在流图中确定循环
  - 全局优化：到达-定值数据流分析/UD链；活跃变量数据流分析/DU链

练习

- 下图是包含7个基本块的流图，其中B1为入口，B7为出口



练习

(1) 对该流图进行到达-定值数据流分析，假设B1的IN信息为ø，请将迭代结束时的结果填充在下表中；

(2) 指出该流图范围内，变量a在(11)的UD链；

	GEN	KILL	IN	OUT
B1			∅	
B2				
B3				
B4				
B5				
B6				
B7				

练习

(3) 对该流图进行活跃变量数据流分析，假设B7的LiveOut信息为ø，请将迭代结束时的结果填充在下表中;

(4) 指出该流图范围内，变量c在(2)的DU链。

	LiveUse	DEF	LiveIn	LiveOut
B1				
B2				
B3				
B4				
B5				
B6				
B7				∅